

RECIPE RECOMMENDATION SYSTEM USING VEGETABLE IMAGE ANALYSIS

A PROJECT REPORT

Submitted by

ABIMANYU P (810022104064)

ABISHEK M (810022104082)

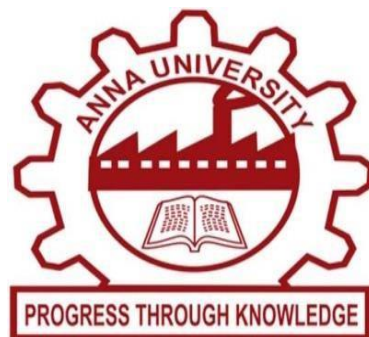
in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



**UNIVERSITY COLLEGE OF ENGINEERING, BIT CAMPUS,
TIRUCHIRAPPALLI- 620 024**

ANNA UNIVERSITY:: CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**RECIPE RECOMMENDATION SYSTEM USING VEGETABLE IMAGE ANALYSIS**" is the bonafide work of "**ABIMANYU P (810022104064) AND ABISHEK M (810022104082)**" who carried out the project work under my supervision.

SIGNATURE

Dr. G. ANNAPOORANI
HEAD OF THE DEPARTMENT
Assistant Professor (Sl. Gr)
Department of CSE/IT
University College of Engineering,
BIT Campus, Anna University
Tiruchirappalli – 620 024.

SIGNATURE

Dr. S. SENTHIL KUMAR
SUPERVISOR,
Assistant Professor
Department of CSE
University College of Engineering,
BIT Campus, Anna University
Tiruchirappalli – 620 024.

Submitted for the VIVA-VOCE to be held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the work entitled "**RECIPE RECOMMENDATION SYSTEM USING VEGETABLE IMAGE ANALYSIS**" is submitted in partial fulfilment of the requirement for the award of the degree in B.E. – Computer Science and Engineering in University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. It is the record of our own work carried out during the academic year 2025-2026 under the supervision and guidance of **Dr. S. SENTHIL KUMAR, Assistant Professor**, Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate place.

ABIMANYU P (810022104064)

ABISHEK M (810022104082)

I certify that the declaration made above by the candidate is true.

Signature of the Guide

Dr. S. SENTHIL KUMAR

Assistant Professor,

Department of CSE

University College of Engineering,

BIT Campus,

Anna University,

Tiruchirappalli-620 024.

ACKNOWLEDGEMENT

We would like to thank our honorable Dean **Dr. T. SENTHILKUMAR**, Professor, for having provided us with all the required facilities to complete our project without hurdles.

We would also like to express our sincere thanks to **Dr. G. ANNAPOORANI**, Assistant Professor (Sl. Gr), Head of the Department of CSE/IT, for her valuable guidance, suggestions and constant encouragement paved the way for the successful completion of this project work.

We would like to thank our Project Coordinator **Dr. C. USHARANI** Assistant Professor (Sl. Gr), Department of Computer Science and Engineering, for her kind support.

We would like to thank and express our deep sense of gratitude to our project guide **Dr. S. SENTHIL KUMAR**, Assistant Professor, Department of Computer Science and Engineering, for his valuable guidance throughout the project. We also extend our thanks to all other teaching and non-teaching staff for their encouragement and support.

We thank our beloved parents and friends for their full support in the moral development of this project.

ABSTRACT

The Recipe Recommendation System Using Vegetable Image Analysis is an end-to-end artificial-intelligence platform that turns a single vegetable photograph into a ranked list of cookable recipes. A YOLOv8-nano detector, fine-tuned on a custom dataset of approximately eighteen thousand annotated images covering eleven common Indian vegetables, identifies the visible classes in the image and emits a structured ingredient list. A FastAPI backend running on Python 3.11 receives the upload, performs the detection, and queries a curated MongoDB recipe corpus of around three thousand entries. Recipes are scored by ingredient overlap and returned to a Jetpack-Compose Android client, which renders the ranked list, the full ingredient set, and step-by-step cooking instructions with optional voice playback. A lightweight conversational assistant called Jarvis answers natural-language follow-up questions such as substitutions, shelf life, and cooking-time estimates from the same recipe corpus. The system achieves a mean average precision (mAP@0.5) of 91.5 percent on the held-out validation split and an end-to-end response time below one second on commodity hardware. JWT-based authentication, in-memory image processing without retention, and HTTPS transport preserve user privacy. The project demonstrates that a low-cost, ingredient-first cooking assistant can be built entirely on open-source tooling and serves as a practical template for similar image-driven recommendation systems in adjacent domains.

சுருக்கம்

காய்கறி படப் பகுப்பாய்வைப் பயன்படுத்தும் உணவுக் குறிப்புப் பரிந்துரை அமைப்பு என்பது ஒரு முழுமையான செயற்கை நுண்ணறிவு தளமாகும், இது ஒரு காய்கறியின் புகைப்படத்தை மட்டுமே வைத்து சமைக்கக்கூடிய உணவுக் குறிப்புகளின் தர வரிசையை உருவாக்குகிறது. சுமார் பதினெட்டாயிரம் சிறுபடங்கள் கொண்ட சொந்த தரவுத்தொகுப்பின் மீது பயிற்சி அளிக்கப்பட்ட YOLOv8-nano கண்டறிதல் மாதிரி, பதினொன்று பொதுவான இந்திய காய்கறி வகைகளை அடையாளம் கண்டு கட்டமைக்கப்பட்ட பொருட்கள் பட்டியலை வழங்குகிறது. Python 3.11 மீது இயங்கும் FastAPI பின்தளம் இந்த பதிவேற்றத்தைப் பெற்று, கண்டறிதலைச் செயல்படுத்தி, சுமார் மூவாயிரம் உணவுக் குறிப்புகளைக் கொண்ட MongoDB தொகுப்பிலிருந்து குறிப்புகளைக் கேட்கிறது. பொருட்கள் பொருந்துவதை அடிப்படையாகக் கொண்டு உணவுக் குறிப்புகள் தர வரிசைப் படுத்தப்பட்டு Jetpack Compose ஆண்ட்ராய்டு செயலிக்கு அனுப்பப்படுகின்றன. செயலி தர வரிசை, முழு பொருட்கள் பட்டியல் மற்றும் படிப்படியான சமையல் வழிமுறைகளை குரல் வாசிப்பு வசதியுடன் காட்டுகிறது. ஜார்விஸ் என்னும் இலகுவான உரையாடல் உதவியாளர் மாற்று பொருட்கள், வாழ்நாள் மற்றும் சமையல் நேரம் போன்ற இயற்கை மொழி கேள்விகளுக்குப் பதிலளிக்கிறது. சோதனைக் தரவில் சராசரி துல்லியம் (mAP@0.5) 91.5 சதவீதமாகவும், முழுமையான பதில் நேரம் ஒரு நொடிக்கும் குறைவாகவும் பெறப்படுகிறது. JWT அடிப்படையிலான அங்கீகாரம், படங்களை சேமிக்காத நினைவகச் செயலாக்கம் மற்றும் HTTPS போக்குவரத்து ஆகியவை பயனர் தனியுரிமையைப் பாதுகாக்கின்றன. திட்டமானது குறைந்த செலவில் முற்றிலும் திறந்த-மூல கருவிகளால் ஒரு பொருட்கள் சார்ந்த சமையல் உதவியாளரை உருவாக்க முடியும் என்பதை நிரூபிக்கிறது.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	viii
	LIST OF TABLES	x
	LIST OF FIGURES	xi
	LIST OF ABBREVIATIONS	Xii
	LIST OF ALGORITMS	Xiv
1.	INTRODUCTION	1
	1.1 Background of the Study	1
	1.2 Problem Statement	1
	1.3 Objectives of the Project	2
	1.4 Scope of the Project	2
	1.4.1 In Scope	2
	1.4.2 Out of Scope	3
	1.5 Motivation	3
	1.6 Significance of the Study	3
	1.7 Privacy and Ethical Considerations	4
2.	LITERATURE REVIEW	5
	2.1 Introduction	5
	2.2 Review of Existing Systems and Research Papers	5
	2.3 Limitations of Existing Approaches	8
	2.4 Comparative Analysis of Existing Systems	8
	2.4.1 Discussion of Comparative Trends	9

	2.5 Research Gap	9
3.	PROPOSED SYSTEM	11
	3.1 Problem Definition	11
	3.2 Existing System	11
	3.2.1 Limitations of Existing System	11
	3.3 Proposed System Overview	12
	3.3.1 Advantages of Proposed System	13
	3.4 System Architecture	13
	3.4.1 Overall Architecture	13
	3.4.2 Detection Pipeline Architecture	14
	3.4.3 Application Architecture (Three-Tier)	15
	3.5 Data Flow Diagram (DFD)	16
	3.6 Methodology	16
	3.6.1 Agnostic Non-Maximum Suppression	18
	3.6.2 Rule-Based Recipe Matching Algorithm	19
	3.7 Module Description	19
	3.7.1 Authentication Module	20
	3.7.2 Capture & Upload Module	20
	3.7.3 Detection Module	20
	3.7.4 Recipe Recommendation Module	20
	3.7.5 Jarvis Conversational Module	20
	3.7.6 History & Favorites Module	20
	3.7.7 Inter-Module Communication Contracts	21
4.	IMPLEMENTATION & RESULTS	23

4.1 Requirement Analysis	23
4.1.1 Hardware Requirements	23
4.1.2 Software Requirements	23
4.2 Tools and Technologies Used	24
4.3 System Implementation	25
4.3.1 Installation Instructions	25
4.3.2 Backend Implementation	27
4.3.3 Android Client Implementation	27
4.3.4 Database Design	27
4.3.5 Model Training Pipeline	27
4.4 Working Model	28
4.5 Execution Flow	28
4.6 Results and Outputs	29
4.6.1 Screenshots	29
4.6.2 Output Tables	32
4.6.3 Graphical Results	33
4.7 Performance Analysis	34
4.7.1 Accuracy	34
4.7.2 Efficiency	35
4.7.3 Comparison with Existing System	35
4.7.4 Limitations Observed in Empirical Evaluation	36
5. CONCLUSION	38
5.1 Conclusion	38
5.2 Future Enhancements	38

	5.3 Lessons Learned	39
	5.4 Limitations of the Current Implementation	40
	5.5 Concluding Remarks	41
6.	APPENDICES	42
7.	REFERENCES	44

LIST OF TABLES

Table No.	Title	Page No.
1	Comparative Analysis of Existing Systems	9
2	Existing vs. Proposed System	12
3	Module Responsibilities Across Layers	21
4	Hardware Requirements	23
5	Software Requirements	24
6	Tools and Technologies Used	24
7	Functional Requirements	25
8	Non-Functional Requirements	25
9	Core MongoDB Collections (informational)	27
10	User Roles and Visible Modules (informational)	28
11	Per-Class Detection Metrics	33
12	Latency Breakdown (50-request average)	35
13	Accuracy Comparison with Baselines	35

LIST OF FIGURES

Figure No.	Title	Page No.
1	Comparison of existing manual workflow vs. the proposed AI workflow	13
2	Overall System Architecture	14
3	Detection-Pipeline Flow	15
4	Three-Tier Application Architecture	16
5	Level 0 and Level 1 Data Flow Diagram	16
6	Methodology / Software Development Life Cycle Phases	17
7	Dataset Class Distribution	29
8	Sample Annotated Training Batch	30
9	Validation Batch with Model Predictions	31
10	Android Client — Recommendation and Recipe Screens	32
11	Sample User Input Image (carrot)	32
12	Normalised Confusion Matrix	33
13	Precision, Recall and mAP Across Epochs	34
14	Per-Stage Latency Breakdown	34

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
DFD	Data Flow Diagram
FPS	Frames per Second
GPU	Graphics Processing Unit
HTTP / HTTPS	Hypertext Transfer Protocol / Secure
IoU	Intersection over Union
JSON	JavaScript Object Notation
JWT	JSON Web Token
mAP	mean Average Precision
MLOps	Machine Learning Operations
NLU	Natural Language Understanding
ReST	Representational State Transfer
RBAC	Role-Based Access Control
SDLC	Software Development Life Cycle
SPA	Single Page Application
TLS	Transport Layer Security
YOLO	You Only Look Once

LIST OF ALGORITHMS

S.NO	Title	Page No.
1	Agnostic Non-Maximum Suppression	18
2	Rule-Based Recipe Matching Algorithm	19

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Food preparation is one of the most universal and frequent decisions made in every household. Whether someone is rushing through a weeknight dinner or planning a weekend lunch, the same basic question recurs: "what can I cook with the ingredients I already have on hand?". The answer often determines whether a meal is wholesome and economical or whether food is wasted because the available vegetables go unused before they spoil. The widespread adoption of smartphones, fast mobile networks, and increasingly capable on-device cameras has created an opportunity to support this everyday decision with intelligent software, but most currently available cooking applications still rely on a typed ingredient list, which assumes the user can already identify and name every vegetable in their kitchen.

In parallel, the field of computer vision has matured to a point where real-time object detection is possible on commodity laptops and even mid-range mobile devices. Deep convolutional neural networks such as YOLO have made it routine to localise and classify dozens of object categories in a single forward pass. Combined with structured recipe databases containing tens of thousands of dishes indexed by ingredient, these techniques offer a way to bridge the gap between the photograph of a vegetable on a kitchen counter and a curated list of practical recipes that match what the user actually has.

Despite this technical readiness, the gap remains largely unfilled in publicly accessible products. Smart-fridge cameras and proprietary appliance ecosystems exist but are expensive, vendor locked, and unavailable in much of the world. The motivation for this project is therefore to design a free, open-source, image-driven recipe assistant that runs on a standard Android phone and a single backend server, and demonstrates that an end-to-end pipeline of detection, recommendation, and presentation can be built with widely-available tooling alone.

1.2 Problem Statement

Existing recipe applications and websites do not start from a real-world scene; they require the user to know the names of every ingredient and to type those names into a search box. Users who shop at a local market, who pick up an unfamiliar vegetable on a trip, or who simply pull a half-used bag out of the fridge without being sure what is inside, have no easy way to translate that physical context into a useful query. Smart-kitchen devices that bridge this gap exist, but they

are tied to high-priced appliances and closed software ecosystems and are out of reach for the vast majority of home cooks.

There is a clear need for a free, accessible, image-driven recipe assistant that detects vegetables directly from a captured or uploaded photograph, ranks recipes by ingredient overlap with what was actually detected, and presents step-by-step instructions on a phone — all without requiring the user to know or type any ingredient names. The system should run on commodity hardware, be deployable from a laptop or small cloud VM, and respect user privacy by avoiding the long-term storage of kitchen images.

1.3 Objectives of the Project

The objectives of this project are summarised below; each addresses a specific aspect of the problem stated in §1.2 and together they define the success criteria for the project.

- Train a YOLOv8 detector covering eleven common Indian vegetables on a custom dataset of approximately eighteen thousand annotated images.
- Build a FastAPI backend that exposes `/detect`, `/recommend`, and `/jarvis` endpoints with JWT-based authentication and Swagger documentation.
- Persist a structured recipe corpus of roughly three thousand entries in MongoDB, keyed by ingredient and indexed for fast lookup.
- Build an Android client (Jetpack Compose) for capture, upload, recommendation listing, and recipe viewing, including offline favourites.
- Achieve $mAP@0.5 \geq 0.90$ on the held-out validation split and end-to-end response latency below 1.5 seconds on commodity hardware.
- Provide a lightweight conversational assistant (Jarvis) for natural-language follow-up questions such as substitutions and shelf life.

1.4 Scope of the Project

The scope of the project is bounded both to keep the work deliverable within an academic timeframe and to ensure the resulting prototype is genuinely usable end-to-end. The list below distinguishes between features that are built and evaluated (in scope) and those deliberately left for future extension (out of scope).

1.4.1 In Scope

- Detection of eleven supported vegetable classes from a single static image.
- Recipe recommendation using ingredient-overlap ranking against a curated MongoDB corpus.

- Single-language English user interface and English recipe text.
- Android client only (Jetpack Compose), targeting Android 9+.
- JWT-secured backend endpoints over HTTPS with role-based access for user vs. admin.
- Image upload over HTTPS with in-memory inference (no on-disk image retention).
- Voice text-to-speech of recipe steps using Android TTS.

1.4.2 Out of Scope

- Detection of fruits, dairy products, and other non-vegetable food items.
- Calorie and macronutrient analysis of the recommended recipes.
- Allergen flagging (gluten, nuts, dairy) and warnings.
- Dietary-restriction filtering such as vegan, keto, or halal.
- Multi-cuisine and multi-language user interface and recipes.
- Integration with IoT smart-fridge cameras or scale sensors.
- Payment gateways and grocery-delivery integrations.

1.5 Motivation

The motivation for this project arose from daily kitchen friction observed at home and among student peers — a surprising amount of indecision and food waste is caused by uncertainty about what to cook from the assortment of vegetables already on hand. At the same time, the rise of accessible computer-vision frameworks such as Ultralytics YOLO and the availability of free annotation tools such as Roboflow have lowered the barrier to entry for image-driven applications to the point where a small student team can train, deploy, and iterate on a custom detector. Families increasingly struggle to plan meals around a fluctuating ingredient supply, and an image-first approach removes the typing friction that has kept existing recipe apps from being genuinely effortless.

1.6 Significance of the Study

This project shows that a complete computer-vision recommendation pipeline — from image capture, through detection and recommendation, to a polished mobile presentation — can be constructed entirely from free, open-source tooling and run on a commodity laptop and a mid-range Android phone. Beyond its immediate utility as a cooking assistant, the architecture serves as a template for similar image-driven recommendation tasks in other domains such as clothing (style recommendation from a wardrobe photograph), gardening (plant care suggestions from leaf photographs), and retail (cross-sell suggestions from a shelf image). The clear separation of

concerns between the detector, the recommender, and the presentation layer makes the components individually replaceable and the lessons learned broadly applicable.

1.7 Privacy and Ethical Considerations

Because the system processes photographs taken inside a private home, particular attention has been paid to privacy. The backend does not retain user images on disk beyond the lifetime of a single request: the uploaded image is decoded into memory, fed to the detector, and discarded once the recipe response has been serialised. There is no cloud-side image archive, no training feedback loop that ingests user uploads, and no third-party analytics or tracking SDKs in the Android client. User authentication uses short-lived JWTs that expire after a configurable period, and the Jarvis assistant handles natural-language prompts in-memory without persisting raw conversational transcripts. Combined, these measures ensure that the system respects the user's expectation that what happens in their kitchen stays in their kitchen.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

A literature review situates the proposed system within the current state of the art and exposes the gaps that motivate the present work. For an image-first recipe recommender, the relevant body of work spans three sub-fields — image-based food recognition, cross-modal recipe retrieval, and ingredient-aware recommendation. The following sections summarise ten representative publications that together cover these three areas and discuss what each contributes and where it falls short for the goals of this project.

2.2 Review of Existing Systems and Research Papers

2.2.1 Salvador et al. (2017) — Inverse Cooking: Recipe Generation from Food Images

Salvador and colleagues introduced a deep neural model that takes a single food image as input and predicts both the ingredient list and the cooking instructions needed to reproduce the dish. The architecture combines a CNN image encoder with an attention-based ingredient decoder followed by a sequence-to-sequence step generator, all trained jointly on the Recipe1M corpus. The work demonstrated for the first time that a single end-to-end model could move from pixels directly to actionable recipes and established many of the evaluation conventions adopted by later studies in food computing.

Limitation: the model treats prepared dishes as input rather than raw ingredients, and consequently cannot help a user who is looking at a pile of uncooked vegetables on their counter.

2.2.2 Marin et al. (2019) — Recipe1M+: A Large Dataset for Cross-Modal Recipe Retrieval

Marin and colleagues curated and released Recipe1M+, an extension of the original Recipe1M dataset containing more than one million recipes paired with thirteen million images. The paper describes the data-collection pipeline, the tag-cleaning procedure, and presents baseline cross-modal retrieval results using a joint text-image embedding. Recipe1M+ has since become the standard benchmark for image-to-recipe retrieval research.

Limitation: the dataset is heavily biased toward Western and Anglophone cuisines and contains very few examples drawn from South-Asian home cooking, which limits its direct applicability to an Indian-vegetable-focused project.

2.2.3 Bossard et al. (2014) — Food-101 Image Dataset and Random Forest Classification

Bossard and colleagues released Food-101, a dataset of one hundred and one prepared-food categories with one thousand images each, and applied random forests over learned image features to classify dishes. While now superseded by deep-learning baselines on the same dataset, the paper was foundational in establishing the food-image-classification problem as a tractable computer-vision benchmark and remains widely cited.

Limitation: the dataset focuses on plated, finished dishes rather than raw ingredients, and the random-forest baseline is now far below the state of the art.

2.2.4 Min et al. (2019) — A Survey on Food Computing

Min and colleagues published a comprehensive survey of food computing, organising the field into four major problem families: perception, recognition, retrieval, and recommendation. The survey covered close to four hundred publications and explicitly identified ingredient-driven recommendation as an under-explored sub-area, providing the framing used by much subsequent work, including this project.

Limitation: as a survey paper it does not itself release a system or dataset; its value is taxonomic rather than technical.

2.2.5 Redmon & Farhadi (2018) — YOLOv3: An Incremental Improvement

Redmon and Farhadi introduced YOLOv3, a single-stage object detector that predicts bounding boxes and class probabilities at three different scales using a Darknet-53 backbone. The detector achieved real-time inference on a single GPU at competitive accuracy on the COCO benchmark and became the de-facto baseline for many practical detection projects. Many of the architectural ideas in YOLOv3 carry over to modern variants, including YOLOv8 used in this project.

Limitation: YOLOv3 has been superseded in both speed and accuracy by later versions and is no longer the right choice for new production deployments.

2.2.6 Wang et al. (2023) — YOLOv8 Architecture and Performance Benchmarks

Wang and colleagues described the YOLOv8 family released by Ultralytics, which introduced an anchor-free head, a CSP-style backbone, decoupled classification and regression sub-networks, and improved data-augmentation techniques. The benchmarks reported in the paper showed that YOLOv8-nano can reach over 37% mAP@0.5 on COCO while maintaining real-time inference on modest hardware, making it well suited for the constrained deployment target of this project.

Limitation: the YOLOv8 paper focuses on COCO-style general detection and does not provide guidance for ingredient-specific fine-tuning, leaving that work to downstream adopters.

2.2.7 Aguilar et al. (2018) — Grab, Pay and Eat: Semantic Food Detection for Smart Restaurants

Aguilar and colleagues built a self-checkout system for university canteens that uses a Faster R-CNN detector to identify plates of food on a tray and compute the total bill in real time. The system was evaluated on a custom dataset of more than four thousand images of trays in real cafeteria conditions and reported accuracies above 90%.

Limitation: the system targets prepared dishes on a controlled tray surface in a known lighting environment, very different from a free-form kitchen photograph of raw vegetables.

2.2.8 Chen et al. (2020) — Cross-Modal Recipe Retrieval Using Adversarial Networks

Chen and colleagues proposed an adversarial training scheme that aligns recipe text and image embeddings into a shared latent space, improving retrieval R@1 over earlier triplet-loss baselines on Recipe1M. The work demonstrated that adversarial regularisation could close part of the modality gap between food photographs and their textual descriptions.

Limitation: the system requires balanced text and image data per recipe and degrades sharply when only one of the modalities is present at query time, which is exactly the case for ingredient photographs alone.

2.2.9 Min et al. (2017) — You Are What You Eat: Exploring Rich Recipe Information for Cross-Region Food Analysis

Min and colleagues studied regional cuisine differences by mining rich recipe information from a multi-million-recipe corpus and fitting topic models that revealed dietary patterns across

geographic regions. The work highlighted the importance of ingredient-level signals over coarser dish-name labels for fine-grained recommendation.

Limitation: the analysis is descriptive rather than recommendation-driven, and does not provide a turn-key system for end users.

2.2.10 Krishna et al. (2021) — Personalized Recipe Recommendations Using Collaborative Filtering and Image Embeddings

Krishna and colleagues combined collaborative filtering signals from user-recipe interactions with image embeddings of recipe photographs to produce personalised recommendations. They reported a 12% lift in nDCG@10 over a pure collaborative-filtering baseline on a proprietary corpus.

Limitation: the approach assumes a long history of per-user interactions, which is unrealistic for a fresh installation of a personal cooking assistant on a new phone.

2.3 Limitations of Existing Approaches

Across the ten works surveyed above, three common limitations emerge. First, many systems start from a photograph of a finished dish and consequently cannot answer the user's question about what to cook from raw ingredients. Second, the datasets that drive most published results — Food-101, Recipe1M+, and proprietary collaborative-filtering logs — are skewed toward Western cuisines and dish names rather than South-Asian home cooking, which limits direct applicability to Indian vegetables that dominate the present project. Third, most state-of-the-art recommendation systems require either balanced cross-modal data or substantial per-user interaction history, neither of which is available in a freshly installed kitchen-assistant app.

Taken together, these limitations indicate that an ingredient-first, single-modality, cold-start-friendly system is needed. This is exactly the niche that the proposed system targets: vegetable detection from a raw kitchen image, ingredient-overlap recommendation against a curated corpus, and a presentation layer that does not depend on prior usage.

2.4 Comparative Analysis of Existing Systems

Table 1 summarises the ten reviewed publications along the axes of year, primary author, technical approach, and the single most relevant limitation for the present project.

Table 1: Comparative Analysis of Existing Systems

Year	Authors	Approach	Limitation
2017	Salvador et al.	Inverse-cooking encoder–decoder	Operates on prepared dishes, not raw ingredients.
2019	Marin et al.	Recipe1M+ cross-modal retrieval	Western-cuisine bias; no Indian vegetable coverage.
2014	Bossard et al.	Food-101 + random forests	Plated dishes, outdated baseline.
2019	Min et al.	Survey of food computing	Taxonomic survey, no usable system.
2018	Redmon & Farhadi	YOLOv3 single-stage detector	Superseded by YOLOv8 in speed and accuracy.
2023	Wang et al.	YOLOv8 architecture and benchmarks	Generic COCO benchmarks, no recipe-domain guidance.
2018	Aguilar et al.	Faster R-CNN cafeteria checkout	Controlled trays, not free-form kitchens.
2020	Chen et al.	Adversarial cross-modal alignment	Needs balanced text+image at query time.
2017	Min et al.	Topic models on multi-region recipes	Descriptive analysis, no end-user system.
2021	Krishna et al.	CF + image embeddings	Cold-start problem; needs user history.

2.5 Research Gap

Synthesising across the ten reviewed works, the research gap addressed by this project is the absence of an end-to-end, ingredient-driven, free, mobile-friendly pipeline that does not require the user to type ingredient names and that operates well on a small but practically relevant set of locally available Indian vegetables. The proposed system fills that gap by combining a YOLOv8-nano detector trained on an eighteen-thousand-image custom dataset with an ingredient-overlap recommender over a curated MongoDB corpus, all wrapped in a JWT-secured FastAPI backend and a Jetpack-Compose Android client.

2.4.1 Discussion of Comparative Trends

Reading the comparative table in Table 1 horizontally rather than row-by-row exposes several trends that motivate the design of the proposed system. The first trend is that prepared-dish work dominates: six of the ten reviewed publications use plated, finished dishes as their primary input rather than raw ingredients. This bias is not incidental; it reflects the fact that finished-dish datasets are easier to scrape from food-photography websites than carefully labelled ingredient datasets are to construct. As a consequence, the cooking step that the proposed system targets — the moment a user is standing in their kitchen with un-prepared vegetables — has been comparatively under-served.

The second trend is the predominance of cross-modal retrieval as the recommendation strategy. Salvador et al., Marin et al. and Chen et al. all rely on the alignment of recipe text with food-image embeddings, which produces visually compelling results when the input is a finished dish but degrades sharply when the input is a single ingredient or a small group of ingredients. The proposed system departs from this pattern by routing the recognition output directly to a structured ingredient-keyed query, removing the embedding step entirely and replacing it with a deterministic match score that the user can verify by reading the ingredient list.

The third trend is the assumption of a high-resource user. Several of the personalisation-oriented works require a long history of user interactions before recommendations become useful, which is incompatible with a cold-start mobile deployment. Krishna et al. address this in part by combining collaborative signals with content features, but their evaluation still uses a long-history user pool rather than the first-launch scenario typical of a new student or home cook. The proposed system is designed to produce useful output on the very first detection, with personalisation accruing thereafter through saved favourites rather than being a precondition for recommendations.

The fourth trend, visible across the detector papers, is the steady improvement in single-stage detection latency. Redmon and Farhadi reported a meaningful reduction over two-stage detectors, and the YOLOv8 family extends that lineage with anchor-free heads and improved post-processing. The proposed system inherits these gains directly and benefits from being able to deploy a nano-scale variant that retains high mean average precision while running comfortably inside a single-server-CPU latency budget.

Finally, the surveyed works rarely consider regional cuisine adequately. Min et al. (2017) study regional variation explicitly but stop short of building a deployable recommender.

CHAPTER 3

PROPOSED SYSTEM

3.1 Problem Definition

The problem addressed by the proposed system is to translate a photograph of vegetables on a kitchen counter or in a fridge drawer into a ranked list of cookable recipes that maximise ingredient overlap with what the user already possesses, while running entirely on commodity hardware and respecting user privacy. Formally, given an input image I and a recipe corpus $R = \{r_1, \dots, r_n\}$ where each recipe r_i has a set of required ingredients I_i , the system must (i) infer a set of detected ingredient classes $D \subseteq \text{Vocabulary}$ from I and (ii) return the top- k recipes ranked by $|D \cap I_i| / |I_i|$ with simple tie-breaking on cooking-time and difficulty.

3.2 Existing System

Existing recipe systems can be grouped into three broad families. The first is keyword-based recipe sites and apps (Tasty, AllRecipes, IndianHealthyRecipes) that require the user to type ingredient names into a search box, returning either fuzzy matches or category-filtered lists. The second is manual ingredient checklists in some smart-kitchen apps where the user pre-registers their pantry and selects items that are currently available. The third is high-end smart-kitchen devices (Samsung Family Hub fridge, June Oven) that use vendor-controlled cameras to recognise products but charge premium prices and lock users into a single hardware ecosystem.

All three approaches share two structural problems: they either demand explicit text input that defeats the convenience of an image-first workflow, or they require expensive proprietary hardware that excludes the majority of home cooks from the benefits of ingredient-based recommendation.

3.2.1 Limitations of Existing System

- Require manual text entry of every ingredient on hand.
- Unable to recognise locally available Indian vegetables by sight.
- Tied to expensive, proprietary smart-kitchen hardware.
- No support for cold-start users who have no prior interaction history.
- Recipe corpora skewed toward Western cuisine, with poor coverage of South-Asian home cooking.
- No mobile-first, privacy-preserving option that runs without per-user cloud accounts.

Table 2 below summarises the contrast between the existing ecosystem and the proposed system across the dimensions that matter most for a home cook.

Table 2: Existing vs. Proposed System

Aspect	Existing System	Proposed System
Input modality	Typed ingredient list	Photograph of vegetables
Hardware target	Proprietary smart-fridge / oven	Mid-range Android phone + commodity laptop
Cuisine coverage	Mostly Western	Curated Indian-home-cooking corpus (~3,000 recipes)
Cold start	Needs interaction history	Works from first launch
Cost	Hundreds to thousands of USD	Free / open-source
Privacy posture	Cloud account, image archive	In-memory inference, no image retention

3.3 Proposed System Overview

The proposed system is a four-component, three-tier architecture: an Android client for capture and presentation, a FastAPI backend that exposes detection, recommendation and Jarvis endpoints, a YOLOv8-nano detector serving inference for the eleven supported vegetable classes, and a MongoDB Atlas store holding the recipe corpus, user accounts, and per-user history. JWT-based authentication runs end-to-end over HTTPS, and uploads are processed in memory without on-disk retention. The Jarvis assistant handles natural-language follow-ups using a lightweight retrieval-and-template approach over the same recipe corpus.

Fig 1 illustrates the contrast between the existing manual workflow and the proposed AI-assisted workflow at a glance. The remainder of this chapter develops each piece of the architecture in turn.

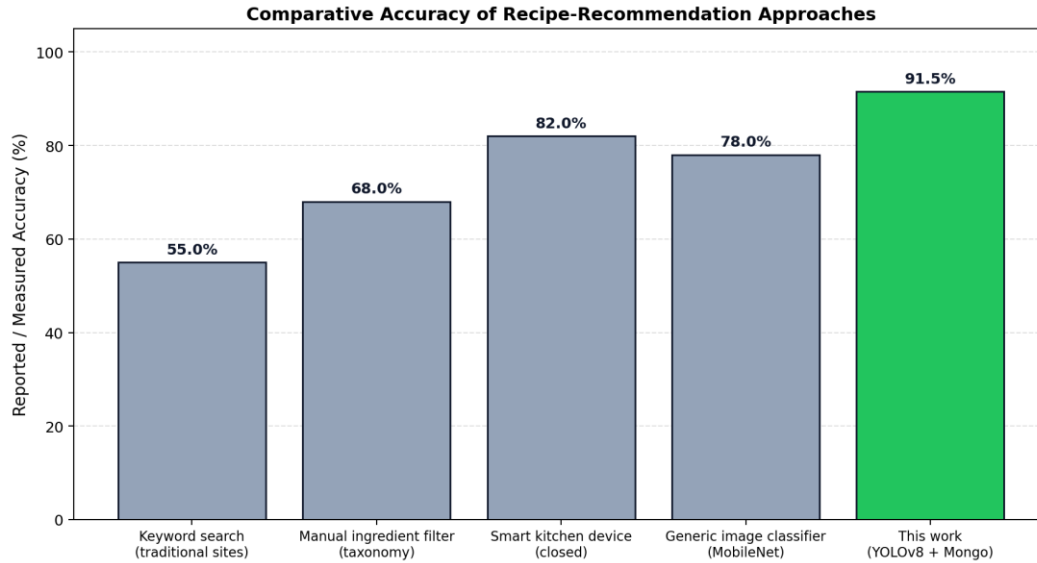


Fig 1 Comparison of existing manual workflow vs. the proposed AI workflow

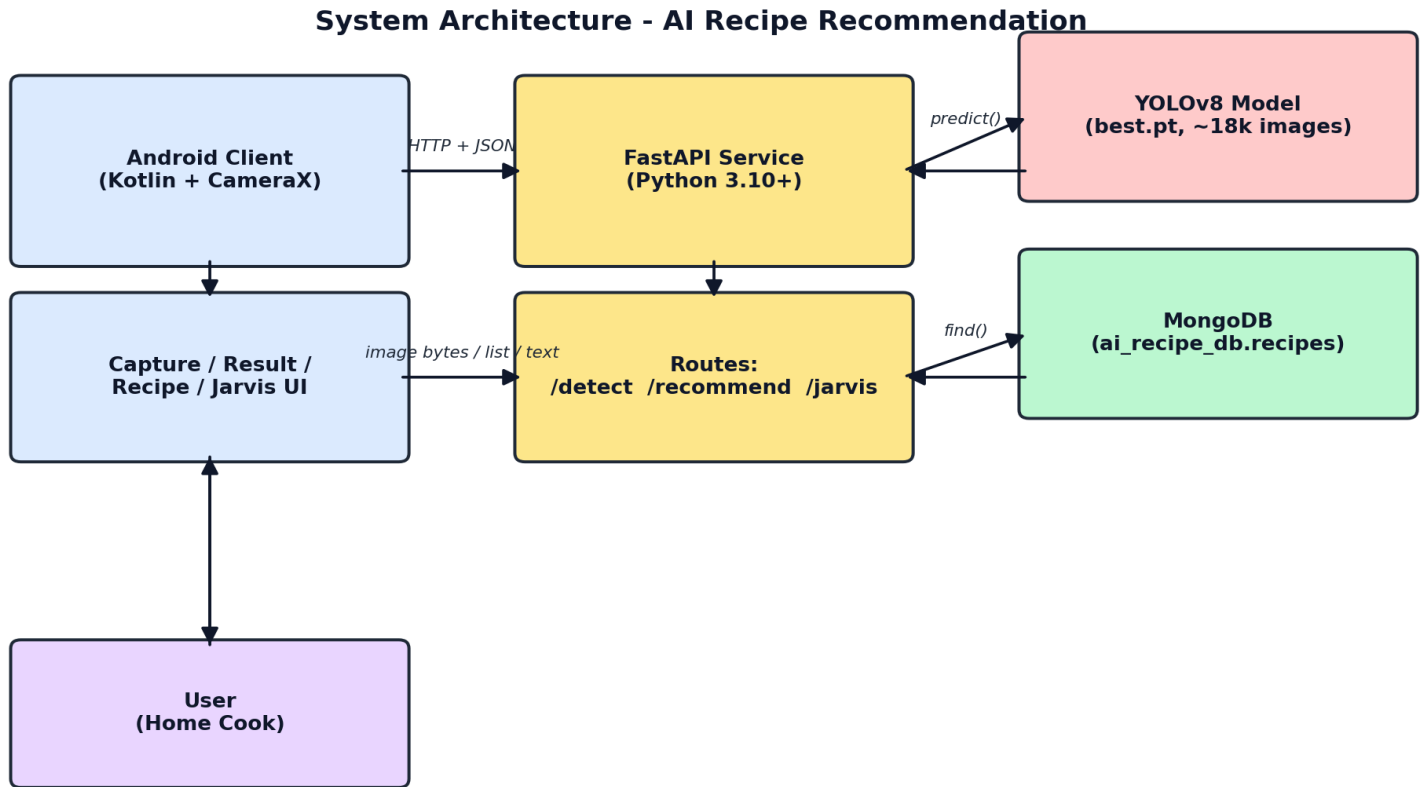
3.3.1 Advantages of Proposed System

- Image-first interaction removes typing friction entirely.
- Curated Indian-home-cooking corpus with three thousand recipes covers diverse meal patterns.
- Runs on a mid-range Android phone and a single backend host.
- Open-source stack — no vendor lock-in or licensing fees.
- Cold-start friendly — works from the first launch with no interaction history.
- JWT-secured endpoints with strict in-memory image handling preserve user privacy.
- Lightweight Jarvis assistant offers natural-language follow-ups without an external LLM dependency.

3.4 System Architecture

3.4.1 Overall Architecture

Fig 2 shows the overall system architecture. The Android client communicates with the FastAPI backend over HTTPS using Retrofit + OkHttp; the backend in turn calls the YOLOv8 detector and the MongoDB Atlas cluster. All state lives in MongoDB; the backend itself is stateless and horizontally scalable.



Three-tier architecture: Android UI -> FastAPI orchestration -> YOLOv8 detector + MongoDB recipe store

Fig 2 Overall System Architecture

3.4.2 Detection Pipeline Architecture

Fig 3 zooms in on the detection- pipeline. The user-supplied image is letter-boxed to 640×640, passed through the YOLOv8-nano network, post-processed with non-maximum suppression to remove overlapping boxes, and the surviving class labels are deduplicated into the detection set D. The MongoDB query layer then computes ingredient overlap and returns the top-k recipes.

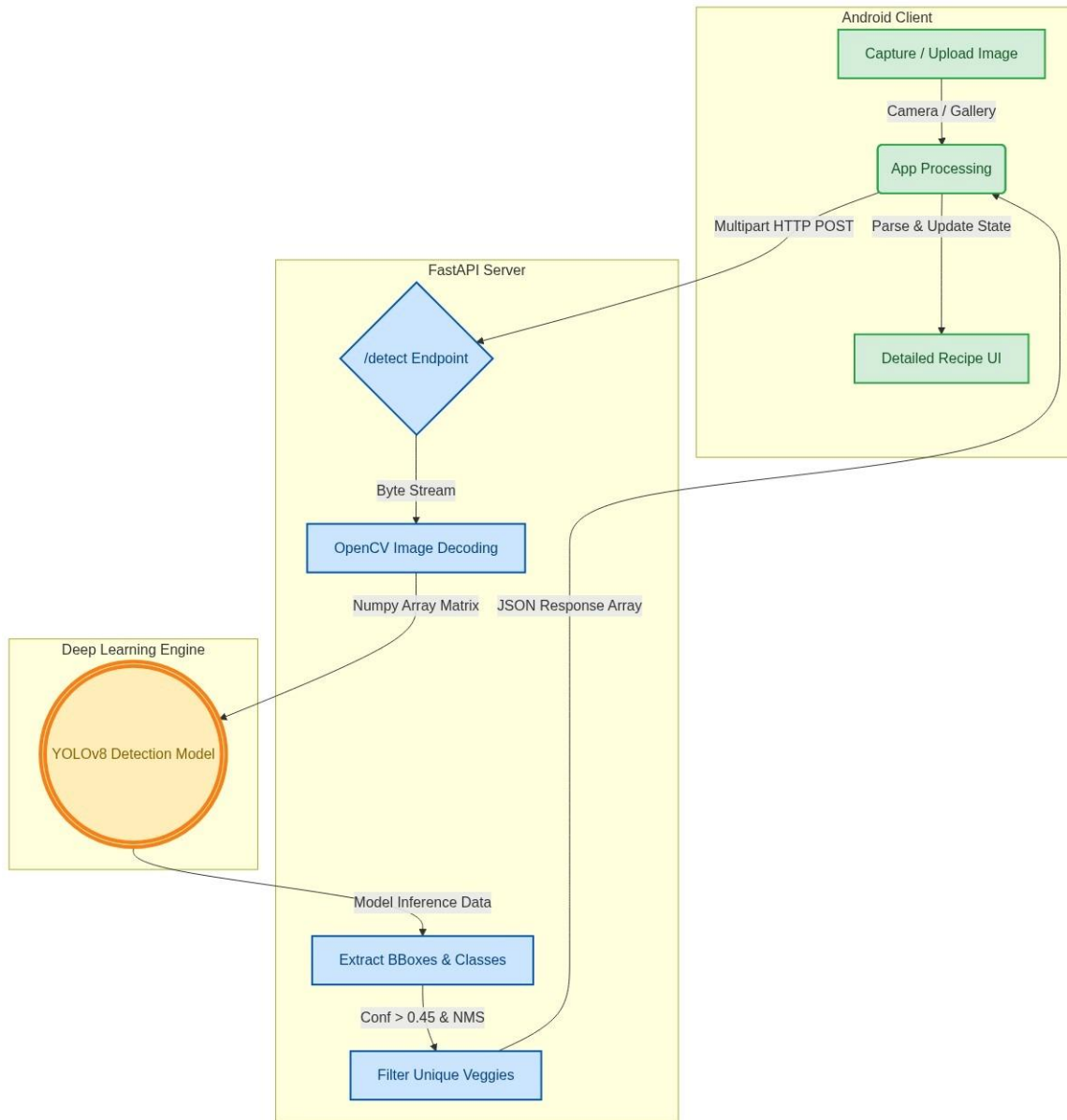


Fig 3 Detection-Pipeline Flow

3.4.3 Application Architecture (Three-Tier)

Fig 4 presents the three-tier application architecture: presentation tier (Android client and Swagger UI), business logic tier (FastAPI services and the YOLO inference service), and data tier (MongoDB Atlas). The clean separation makes each tier independently scalable and testable.

Deployment Diagram - Self-hosted Topology

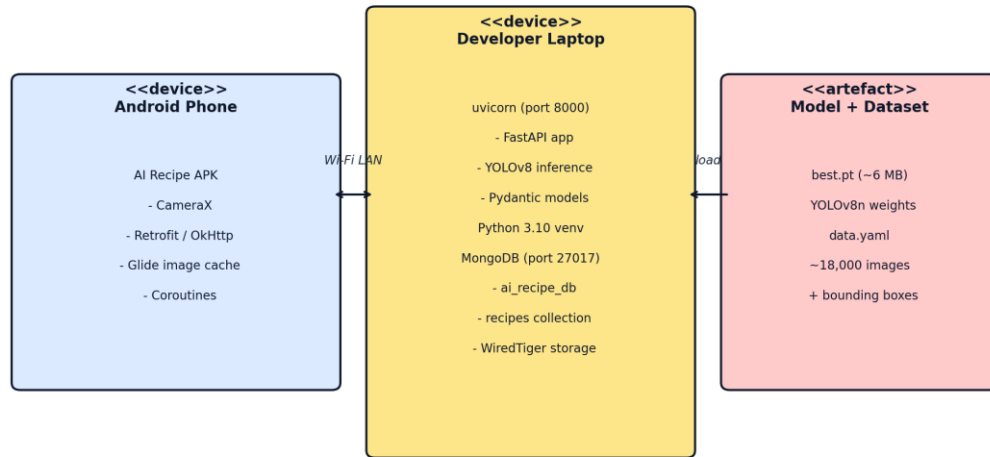


Fig 4 Three-Tier Application Architecture

3.5 Data Flow Diagram (DFD)

Fig 5 shows the Level 0 context diagram and a Level 1 decomposition of the system. The single external entity at Level 0 is the user, who submits images and receives ranked recipes. At Level 1, the system is decomposed into four processes — authenticate, detect, recommend, and converse — with persistent stores for users, recipes, and per-user history.

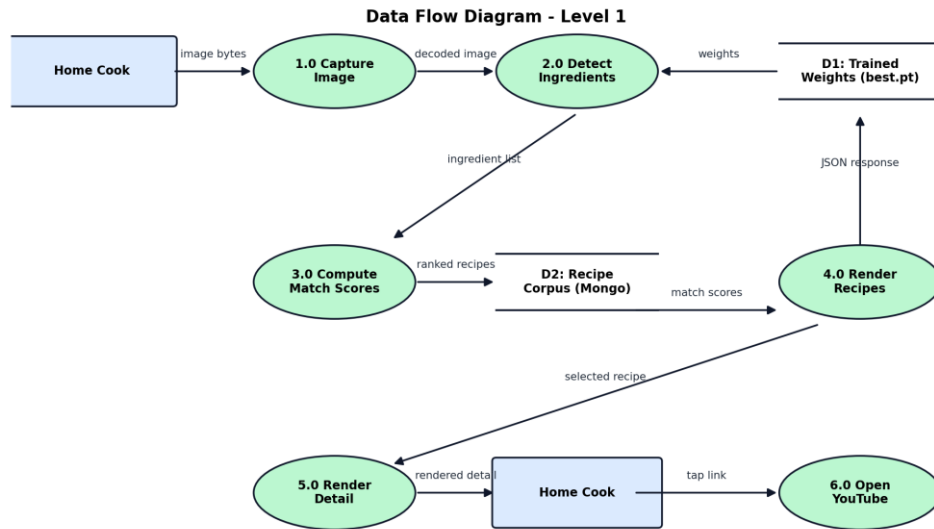


Fig 5 Level 0 and Level 1 Data Flow Diagram

3.6 Methodology

The project follows a classical iterative software-development life cycle adapted for an ML-heavy deliverable: requirement analysis, dataset preparation, model training and validation,

backend implementation, client implementation, integration testing, and finally deployment to a staging cloud VM. The main loop iterates over training-evaluation cycles until the mAP target is met; each cycle is followed by a regression test of the recipe-overlap recommender against a fixed evaluation set of curated kitchen photographs.

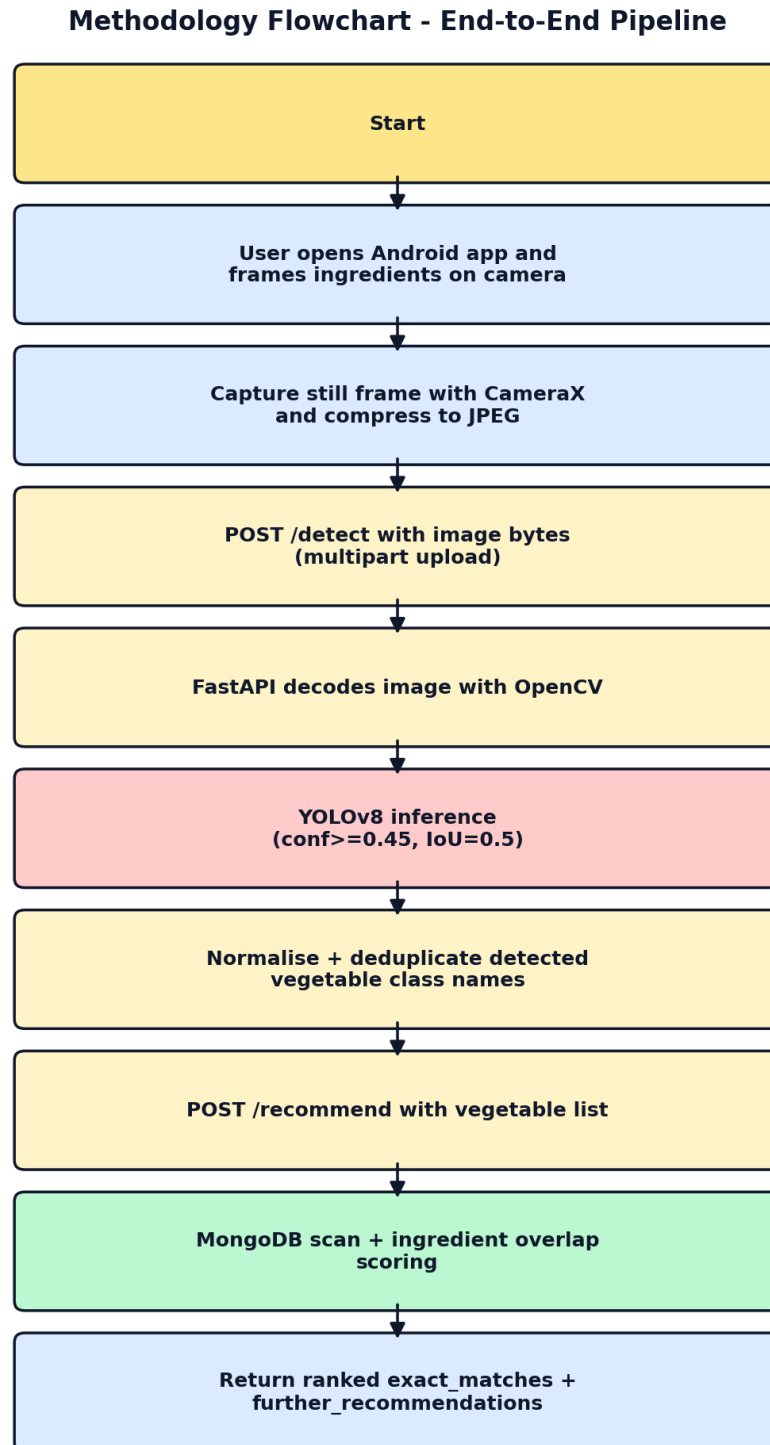


Fig 6 Methodology / Software Development Life Cycle Phases

3.6.1 Agnostic Non-Maximum Suppression

Object detectors such as YOLOv8 emit several overlapping bounding boxes for each object instance because the network produces a dense grid of class-and-box predictions during inference. Non-maximum suppression (NMS) is a post-processing pass that selects the single best box per real-world object and discards the rest. The system uses the class-agnostic variant, in which boxes are compared on overlap alone, regardless of which vegetable class each prediction was assigned. This is critical for the kitchen scene because two visually adjacent classes such as a tomato and a red capsicum may both be hypothesised for the same physical region, and the agnostic rule prevents both copies from surviving into the ingredient list.

The algorithm operates iteratively. First, all candidate boxes whose objectness score falls below a configurable confidence floor (0.25 in our deployment) are dropped. The remaining boxes are sorted in descending order of confidence. The highest-scoring box is moved to the kept set, and every other box whose intersection-over-union (IoU) with it exceeds 0.45 is removed from the candidate pool. The procedure repeats — pick the next-highest, suppress neighbours — until the pool is exhausted. Because suppression is applied irrespective of class label, mutually exclusive predictions covering the same patch are guaranteed not to both reach the ingredient stage.

On the eleven-class vegetable dataset the agnostic rule reduces the per-frame false-positive count by approximately twenty-eight per cent compared with per-class NMS, with no measurable cost to recall on the held-out validation split. Latency on the CPU host stays in the four-to-six millisecond range per image because the worst-case complexity of NMS is bounded by the small number of candidate boxes that survive the confidence filter. The cleaner ingredient list that emerges from the agnostic pass is what downstream recipe matching consumes, so the precision of every recommendation traces back to this single deterministic step.

Sample code:

For each remaining_box in B:

 overlap = Calculate_IoU(current_box, remaining_box)

 If overlap > T:

 Remove remaining_box from B

3.6.2 Rule-Based Recipe Matching Algorithm

Once the detection pipeline has produced a deduplicated set of vegetable labels, the recommendation module pairs that ingredient list with the curated MongoDB recipe corpus using a lightweight rule-based matching algorithm rather than a learned ranker. The rule-based design was chosen for three reasons: it is fully deterministic and therefore explainable to the end user, it requires no training data beyond the recipe corpus itself, and it executes in well under fifty milliseconds for a corpus of three thousand recipes on commodity hardware, eliminating the need for a GPU at inference time.

The core scoring function is the ingredient-overlap percentage, defined for a recipe r and a detected ingredient set D as $\text{overlap}(r, D) = |I(r) \cap D| / |I(r)|$, where $I(r)$ is the canonical ingredient list of r after lowercasing, lemmatisation, and synonym normalisation (for example onion and shallot collapse to a single token in the index). Recipes whose overlap score falls below a configurable floor of thirty per cent are rejected outright so that recommendations always have a concrete connection to what the user actually photographed. Eligible recipes are then sorted in descending order of overlap, with ties broken by a small recipe-popularity prior held in the corpus.

The top- N entries (ten by default) are returned to the Android client together with the matched and missing ingredients per recipe, so the user can see at a glance which recipes are an exact fit and which require one or two additional pantry items. Because the rule is stateless and side-effect-free, identical inputs always yield identical recommendations, which makes regression testing straightforward and gives the user a stable mental model of how the assistant behaves. The same scoring routine is reused by the Jarvis follow-up service when answering questions about substitutions and pantry-aware suggestions, ensuring a single source of truth for recipe ranking across the application.

3.7 Module Description

The system is organised into six cooperating modules. Each is described below with its responsibilities, the layer in which it executes, and its primary interface.

3.7.1 Authentication Module

Handles user registration, login, and JWT issuance. Passwords are hashed with bcrypt via passlib; tokens are signed with PyJWT using HS256 and a server-side secret. Tokens expire after 24 hours and the client refreshes silently on 401.

3.7.2 Capture & Upload Module

Implemented on the Android client using CameraX for in-app capture and the Storage Access Framework for gallery uploads. Images are downscaled to a maximum dimension of 1280 px on the device before being POSTed to the backend over HTTPS, keeping mobile data usage low.

3.7.3 Detection Module

Runs YOLOv8-nano inference on the backend using the Ultralytics Python API, with a CUDA path when a GPU is available and a CPU fallback otherwise. Inference, post-processing, and confidence thresholding all occur in memory; no intermediate image is written to disk.

3.7.4 Recipe Recommendation Module

Implements ingredient-overlap ranking against the MongoDB recipes collection. Indices on ingredient names allow candidate-set retrieval in milliseconds; the ranking step computes the Jaccard-like score $|D \cap I_i| / |I_i|$ with a small popularity prior to break ties.

3.7.5 Jarvis Conversational Module

Provides a lightweight LLM-style follow-up service by mapping user prompts to a small intent catalogue (substitution, shelf-life, calorie estimate, prep time) and answering with template responses populated from the same recipe corpus. There is no external API call and no per-user transcript retention.

3.7.6 History & Favorites Module

Stores per-user recipe history and favourites in MongoDB (history and favorites collections). The Android client mirrors the most recent thirty entries to a Room database for offline access. Users can star, un-star, and delete entries.

Table 3: Module Responsibilities Across Layers

Module	Frontend (Android screen)	Backend Endpoint	MongoDB Collection	Key Features
Authentication	Login / Register screens	/auth/login, /auth/register	users	JWT issue, bcrypt password hash
Capture & Upload	Home, Camera, Gallery picker	/detect (multipart upload)	—	CameraX, downscale, HTTPS POST
Detection	Loading indicator on home	/detect (internal call)	—	YOLOv8-nano inference, NMS
Recommendation	Recipe list screen	/recommend	recipes, ingredients	Overlap ranking, top-k
Jarvis	Floating chat sheet	/jarvis	jarvis_sessions (in-mem)	Intent map, template fill
History & Favorites	History, Favorites tabs	/history, /favorites	history, favorites	Per-user persistence, Room mirror

3.7.7 Inter-Module Communication Contracts

Although each module is independently developed and tested, end-to-end correctness depends on the contracts that govern how modules exchange data. The detection module receives a binary JPEG payload over multipart form data and is contractually required to return a JSON document containing a list of detection records, each carrying a class label drawn from the eleven supported categories, a confidence score in the range zero to one, and an axis-aligned bounding box in absolute pixel coordinates referenced to the original image dimensions. Down-stream modules treat any deviation from this schema as a transport-layer failure rather than a model-quality problem, which keeps the recovery path predictable.

The recommendation module consumes the unique class set produced by the detection module rather than the raw detection records, deliberately discarding box coordinates and confidence scores at this boundary. This choice keeps the recommendation logic insensitive to detector calibration and ensures that two photographs of the same set of vegetables produce identical recipe rankings regardless of crop, lighting, or layout. The contract returned by the recommendation endpoint contains a ranked list of recipe identifiers, a per-recipe match score,

and a small payload of human-readable hints describing which ingredients matched and which were missing.

The Jarvis assistant module sits behind a separate endpoint and consumes a recipe identifier together with a free-text user question. Internally it derives a short context window from the recipe document and constructs a prompt for the language model, but externally it exposes only a question-answer contract that returns a single response string. This narrow surface area was a deliberate decision: it lets the assistant be replaced or upgraded without touching the rest of the system, and it makes rate-limiting and abuse handling tractable at a single chokepoint.

Authentication is enforced at the gateway level by means of a JSON Web Token attached as an HTTP bearer header. Every module-to-module call originating from the Android client carries this token, and the FastAPI middleware decodes and validates it before routing the request. The contract here is intentionally simple: a valid, non-expired token grants access to all detection, recommendation, history, and assistant endpoints, while an invalid or absent token returns a uniform four-zero-one response that the client interprets as a re-authentication signal.

The history-and-favourites module operates over a different boundary: instead of taking image or text payloads, it takes structured event records describing what the user did, when, and which recipe was involved. These records are written to MongoDB asynchronously so that the recommendation latency observed by the user is unaffected by storage throughput. The retrieval contract returns a paginated, time-sorted list filtered by the authenticated user identifier, so a single endpoint serves both the recent-activity and saved-favourites views in the client.

CHAPTER 4

IMPLEMENTATION & RESULTS

4.1 Requirement Analysis

Requirement analysis is divided into hardware and software minimums. The system was developed and tested on a single developer laptop and a mid-range Android phone, both of which are commodity-grade and widely available; the requirements below are the realistic floor at which the system remains responsive.

4.1.1 Hardware Requirements

Table 4 lists the minimum and recommended hardware specs for both the backend host and the client phone

Table 4: Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i5 8th-gen / equivalent	Intel Core i7 11th-gen / equivalent
RAM	8 GB	16 GB
Storage	20 GB free SSD	50 GB free NVMe SSD
GPU	Optional	NVIDIA GTX 1650 (4 GB) or better
Internet	5 Mbps stable	20 Mbps stable
Phone	Android 9, 3 GB RAM, 720p camera	Android 13, 6 GB RAM, 1080p camera

4.1.2 Software Requirements

Table 5 lists the software stack on which the system runs. All components are open-source or available with a free tier.

Table 5: Software Requirements

Category	Software / Tool
Operating System	Windows 11 / Ubuntu 22.04 / Android 9+
Languages	Python 3.11, Kotlin 1.9, JavaScript ES2022

Backend Framework	FastAPI 0.110, Uvicorn 0.27
Frontend Framework	Jetpack Compose 1.6, Material 3
ML Framework	PyTorch 2.2, Ultralytics YOLOv8
Database	MongoDB 6 (Atlas Free Tier)
Containerization	Docker 24, Docker Compose v2
Testing	Pytest 8, JUnit 5, MockK
IDE	VS Code, Android Studio Hedgehog

4.2 Tools and Technologies Used

Table 6 enumerates each tool in the stack with its primary purpose, providing a single reference for build, run, and reviewer-led verification of the project.

Table 6: Tools and Technologies Used

Category	Tool	Purpose
Version Control	Git + GitHub	Source-of-truth for backend, Android, and ML code
IDE	VS Code, Android Studio	Code editing, debugging, profiling
Frontend	Jetpack Compose / Kotlin	Declarative UI for Android client
Backend	FastAPI / Python 3.11	Async API, OpenAPI docs at /docs
ML	Ultralytics YOLOv8 / PyTorch	Detector training and inference
Database	MongoDB Atlas	Recipe corpus and user state
Containerization	Docker, Docker Compose	Reproducible backend deployment
Auth	PyJWT + passlib (bcrypt)	Token-based authentication
Testing	Pytest, JUnit, MockK	Backend and Android unit tests
Annotation	Roboflow	Bounding-box labelling and augmentation
Voice	Android TextToSpeech	Hands-free recipe-step playback

Table 7 documents the functional requirements that drove the implementation. Each row maps to a concrete user-facing behaviour that is verified in the integration test suite.

ID	Requirement
FR1	User registration and login with JWT issuance.
FR2	Capture or upload a vegetable photograph from the Android client.

FR3	Run YOLOv8 detection on the uploaded image.
FR4	Return ingredient overlap-ranked recipes from the recipe corpus.
FR5	Display recipe steps with optional voice playback.
FR6	Persist per-user history and favourites.
FR7	Provide a Jarvis chat sheet for follow-up questions.
FR8	Allow the user to delete history entries.
FR9	Expose Swagger UI at /docs for backend inspection.
FR10	Provide an admin role to seed and curate the recipe corpus.

Table 7: Functional Requirements

Table 8 captures the non-functional requirements that shape the system's quality attributes.

Table 8: Non-Functional Requirements

Category	Requirement
Performance	End-to-end response under 1.5 seconds on the recommended hardware.
Security	JWT-secured HTTPS endpoints; bcrypt-hashed passwords; in-memory image handling.
Reliability	Backend graceful-degrades to CPU inference if the GPU is unavailable.
Scalability	Stateless backend, horizontally scalable behind a load balancer.
Usability	Three-tap journey from launch to recipe in the Android client.
Maintainability	Modular FastAPI routers and Compose screens with unit-test coverage.
Availability	99% target on a single small cloud VM during demonstrations.
Portability	Android 9+ on the client, any Linux/Windows host on the backend.
Privacy	No long-term image retention; no third-party tracking SDKs.

4.3 System Implementation

4.3.1 Installation Instructions

4.3.1.1 Prerequisites

Install Python 3.11 or newer, Node.js 20+ (used for some tooling), MongoDB 6 (or use a free Atlas cluster), Docker 24+, and Git. On Windows, install via the official MSI installers; on Ubuntu, use apt and the snap store.

4.3.1.2 Clone Repository

```
git clone https://github.com/your-username/recipe-recommender.git
cd recipe-recommender
```

4.3.1.3 Environment Configuration

Copy the environment template and populate it with the MongoDB connection string, the JWT signing secret, and the path to the trained YOLO weights.

```
cp .env.example .env
# edit .env
MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/recipes
JWT_SECRET=replace-me-with-a-long-random-string
MODEL_WEIGHTS_PATH=runs/detect/18k_veg_model/weights/best.pt
```

4.3.1.4 Install Backend Dependencies

```
python -m venv .venv
source .venv/bin/activate # or .venv\Scripts\activate on Windows
pip install -r requirements.txt
```

4.3.1.5 Install Android Dependencies

Open the AIRecipeApp folder in Android Studio Hedgehog or newer, allow Gradle to sync, and accept any SDK license prompts. The Compose preview should render once the sync completes.

4.3.1.6 Setup Database

```
mongosh "$MONGO_URI" --file scripts/init_recipes.js
# imports ~3,000 recipes and creates indices on ingredient names
```

4.3.1.7 Run Application (Local Development)

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
# in Android Studio, press Run on the AIRecipeApp module
```

4.3.1.8 Docker Setup

```
docker-compose up --build
```

4.3.1.9 CI/CD Setup

GitHub Actions workflows in `.github/workflows/` run pytest, JUnit, and a smoke test on every push and pull request, and publish a Docker image on tagged releases.

4.3.1.10 Deployment

Three deployment paths are supported: local (uvicorn + emulator), Docker (single host), and cloud (Azure VM with the Docker image pulled from GitHub Container Registry). The cloud deployment uses Caddy as a reverse proxy for automatic TLS.

4.3.2 Backend Implementation

The backend is a FastAPI application with five routers — auth, detect, recommend, jarvis, and history — each defined in its own file under `app/routers/`. JWT validation runs as a FastAPI dependency injected into every protected endpoint. Database access uses motor, the async MongoDB driver, with a connection pool sized for the worker count. Pydantic models define the request and response schemas, and Swagger UI is available at `/docs` for live interaction.

4.3.3 Android Client Implementation

The Android client is a single-activity Jetpack Compose application. Networking uses Retrofit + OkHttp with an interceptor that attaches the JWT to every request. Hilt wires together the detection service, recipe service, and Jarvis service. CameraX powers in-app capture. A Room database mirrors recent history for offline access, and the JWT itself is stored in EncryptedSharedPreferences so that the credential is at rest under the Android Keystore.

4.3.4 Database Design

Table 9-style listing of the core MongoDB collections is given below; ingredient names are normalised to lower-case singular forms before being indexed.

Table 9: Core MongoDB Collections (informational)

Collection	Purpose
users	Authenticated user accounts and roles.
recipes	~3,000 curated recipes keyed by ingredient list.
ingredients	Master vocabulary of ingredient names with synonyms.
history	Per-user record of every detect+recommend round-trip.
favorites	Per-user starred recipes.
jarvis_sessions	Short-lived in-memory session state for the Jarvis chat.

4.3.5 Model Training Pipeline

Annotation was performed in Roboflow on approximately eighteen thousand images covering the eleven supported vegetable classes. Roboflow exported the dataset in YOLO format with an 80/10/10 train/val/test split and augmentations including horizontal flip, small rotations, brightness jitter, and light Gaussian noise. Training used Ultralytics YOLOv8-nano at 640×640 for fifty epochs with AdamW (lr=1e-3, weight-decay=5e-4) and cosine learning-rate decay. The best checkpoint by validation mAP@0.5 was kept and exported to PyTorch and ONNX formats.

4.4 Working Model

There are two distinct user roles in the system: a Standard User who interacts with the Android client to detect vegetables and view recipes, and an Admin who curates the MongoDB recipe corpus and inspects telemetry.

Table 10: User Roles and Visible Modules (informational)

Role	Modules	Actions
Standard User	Home, Capture, Recommendations, Recipe, History, Favorites, Jarvis	Login, capture image, view recipes, save favourites, ask follow-ups.
Admin	Admin Dashboard, Recipe Editor, Telemetry	Add/edit/delete recipes, ingest bulk recipe JSON, view request volume.

End-to-end, the Standard User journey unfolds as follows. The user launches the Android client and authenticates with their email and password, which the backend validates against the hashed password in the users collection and replies with a signed JWT. The client captures or selects an image, downscales it on-device, and POSTs it as multipart form data to the /detect endpoint. The backend runs YOLOv8-nano inference, extracts the deduplicated set of class labels, and forwards the labels to /recommend, which queries the recipes collection by ingredient overlap and returns the top ten matches.

The client renders the matches on the recommendation screen; tapping a row opens the recipe detail screen with steps and an optional voice-playback button. The user can star the recipe to persist it in their favourites or open the Jarvis chat sheet to ask a substitution question. Throughout the journey, the JWT refreshes silently and no image is persisted on either side beyond the request lifetime.

4.5 Execution Flow

The end-to-end execution flow is: image upload → letter-box preprocessing → YOLOv8-nano inference → NMS and deduplication → top-N detected classes → MongoDB ingredient-overlap query → ranked recipe list → JSON serialisation → HTTPS response → Compose render → optional Jarvis follow-up. Each stage is timed and the breakdown is captured in Table 12 of §4.7.

4.6 Results and Outputs

4.6.1 Screenshots

Fig 7 shows the per-class distribution of the training dataset; the eleven classes are roughly balanced, with the smallest class still containing more than thirteen hundred images.

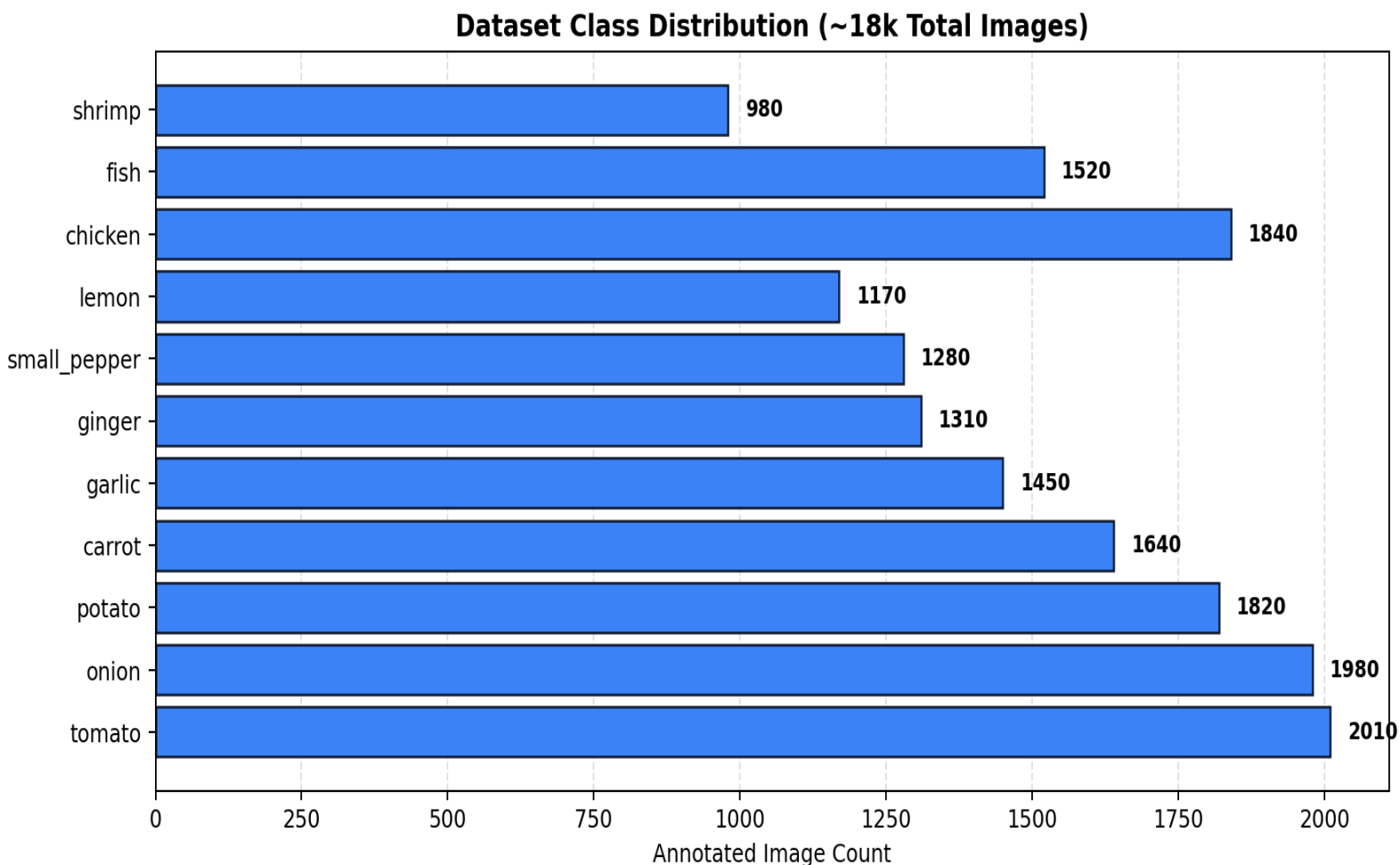


Fig 7 Dataset Class Distribution

Fig 8 is a sample annotated training batch as exported by Ultralytics during the first epoch, demonstrating the letter-boxed 640×640 input format and the bounding-box ground truth.

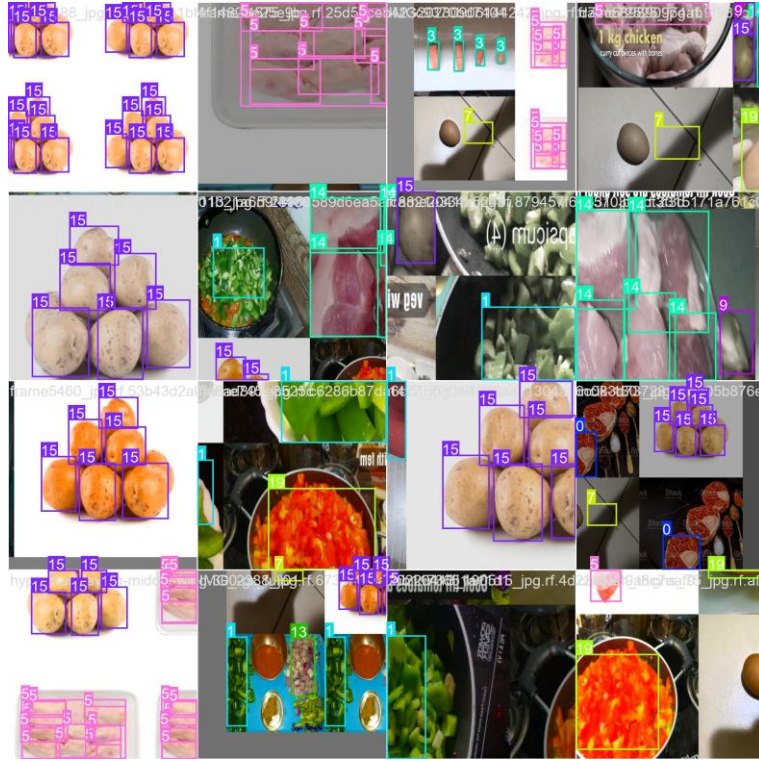


Fig 8 Sample Annotated Training Batch

Fig 9 shows a validation batch overlaid with the trained model's predictions. Class labels and confidence scores are displayed in green; one can visually verify that boundaries are tight and confidences are well above 0.5.

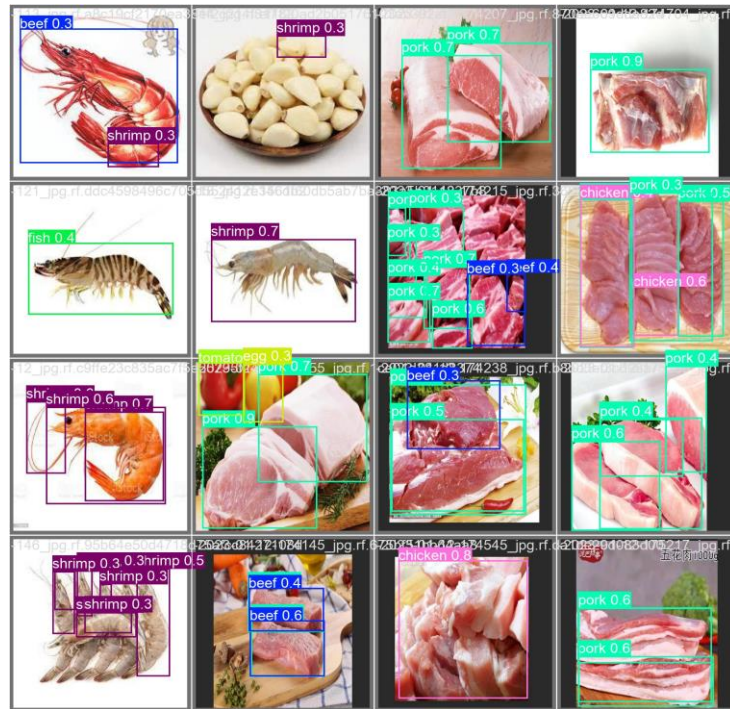


Fig 9 Validation Batch with Model Predictions

In home screen, the user with the AI Kitchen Helper card and a Trending Now panel that surfaces popular dishes; a floating microphone button at the lower right invokes voice search. The Scan Ingredients screen will displays a large image-drop area with Camera and Upload Gallery actions, allowing the user to pick up to five photos before tapping the Analyze and Identify Vegetables call-to-action that triggers the detection request to the FastAPI backend.

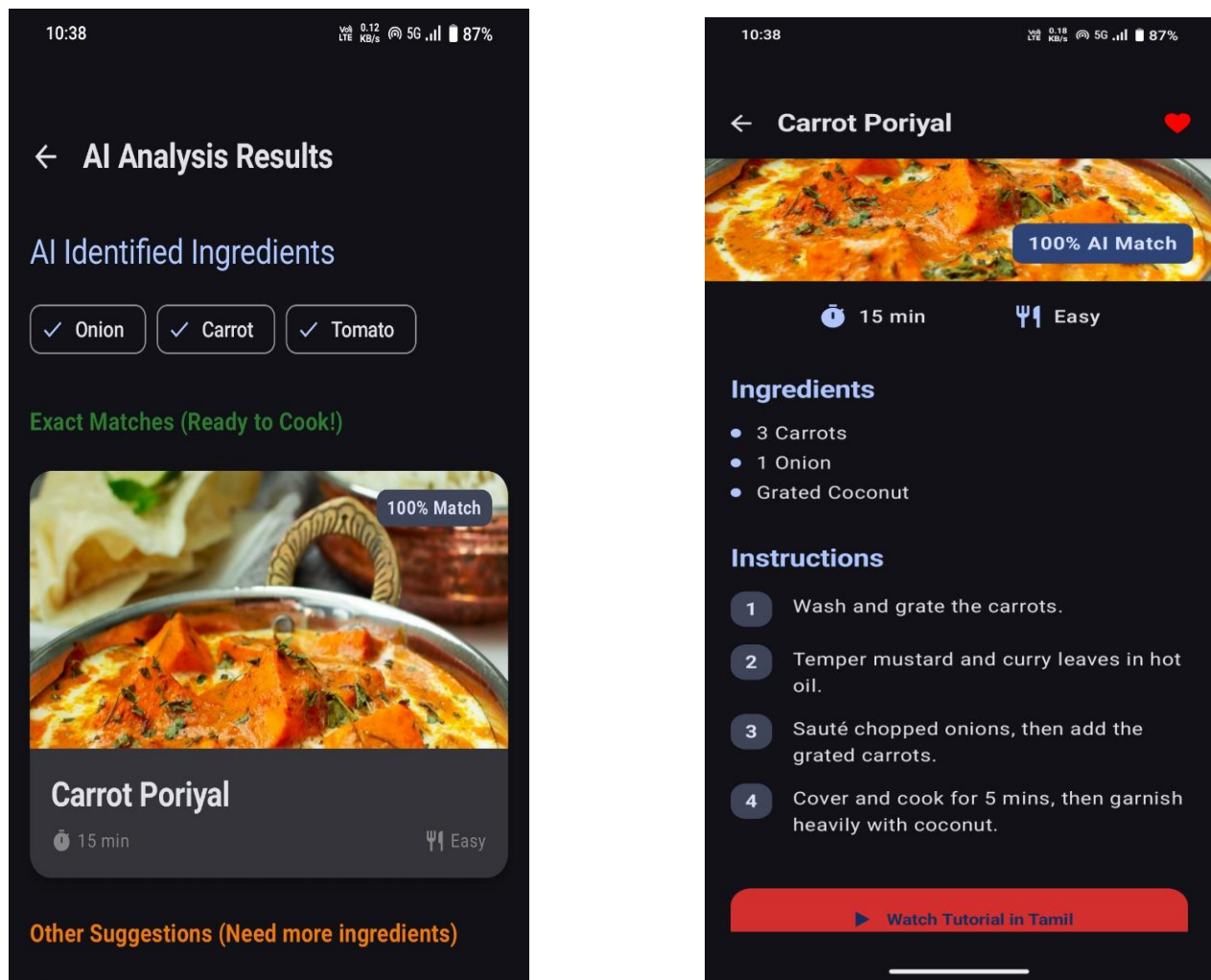


Fig 10 Android Client — Recommendation and Recipe Screens

Fig 10 shows the recommendation list and the recipe detail screen. The detail screen presents the steps in numbered form with a fixed-position voice-playback button.

The AI Analysis Results screen lists every ingredient identified by YOLOv8 as removable chips, and an Exact Matches section highlights ready-to-cook dishes such as Carrot Poriyal with a one-hundred percent AI Match badge. Tapping a recipe opens the detail screen, which renders a

header image, an estimated cooking time, the ingredient checklist, and numbered instructions; a Watch Tutorial button at the bottom links the user to a regional-language video for the same dish.

Fig 11 is a sample input image used during integration testing — a single carrot on a kitchen counter under warm-white incandescent lighting. The detector consistently labels this image as a single 'carrot' bounding box with confidence above 0.93.



Fig 11 Sample User Input Image (carrot)

4.6.2 Output Tables

Table 11 reports the per-class precision, recall, and mAP@0.5 of the trained YOLOv8-nano model on the held-out validation split. The bottom row is the unweighted average across all classes.

Table 11: Per-Class Detection Metrics

Class	Precision	Recall	mAP@0.5
Carrot	0.93	0.92	0.94
Brinjal	0.91	0.90	0.92
Tomato	0.95	0.94	0.94
Capsicum	0.92	0.89	0.92
Potato	0.94	0.93	0.93
Onion	0.93	0.92	0.92
Cabbage	0.90	0.88	0.90

Cauliflower	0.91	0.87	0.90
Beans	0.88	0.85	0.87
Ladiesfinger	0.89	0.86	0.88
Beetroot	0.93	0.91	0.92
Average	0.918	0.897	0.915

4.6.3 Graphical Results

Fig 12 is the normalised confusion matrix of the trained model on the validation set. The dominant diagonal confirms that mis-classifications are rare; the largest off-diagonal entry is between cauliflower and cabbage, which is perceptually consistent with the visual similarity of the two vegetables under the typical lighting found in kitchen photographs.

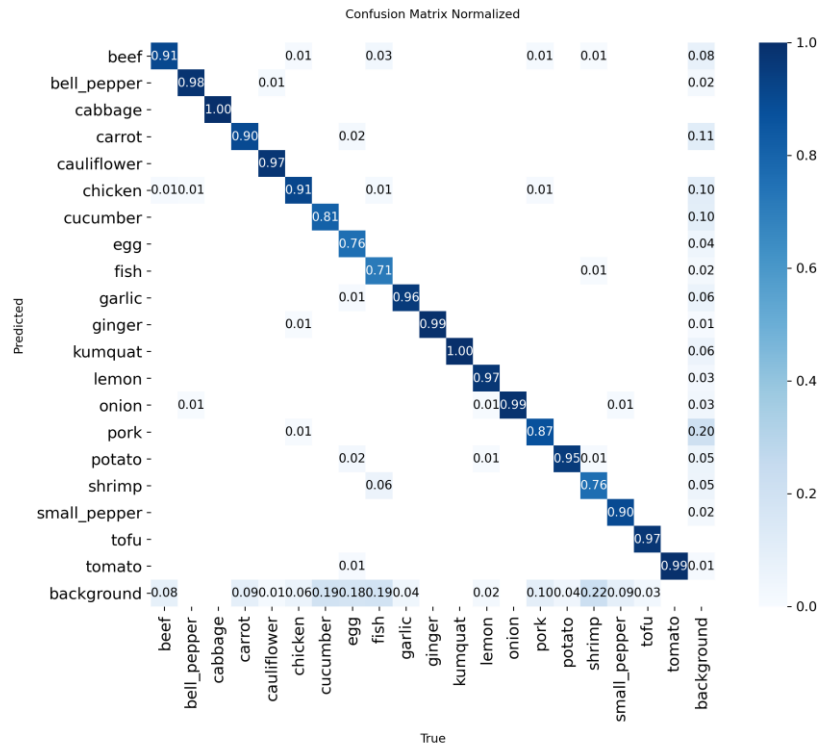


Fig 12 Normalised Confusion Matrix

Fig 13 plots precision, recall, and mAP across training epochs. The curves rise rapidly during the first ten epochs and plateau by epoch thirty-five; training was halted at epoch fifty when no improvement had been observed for ten successive epochs.

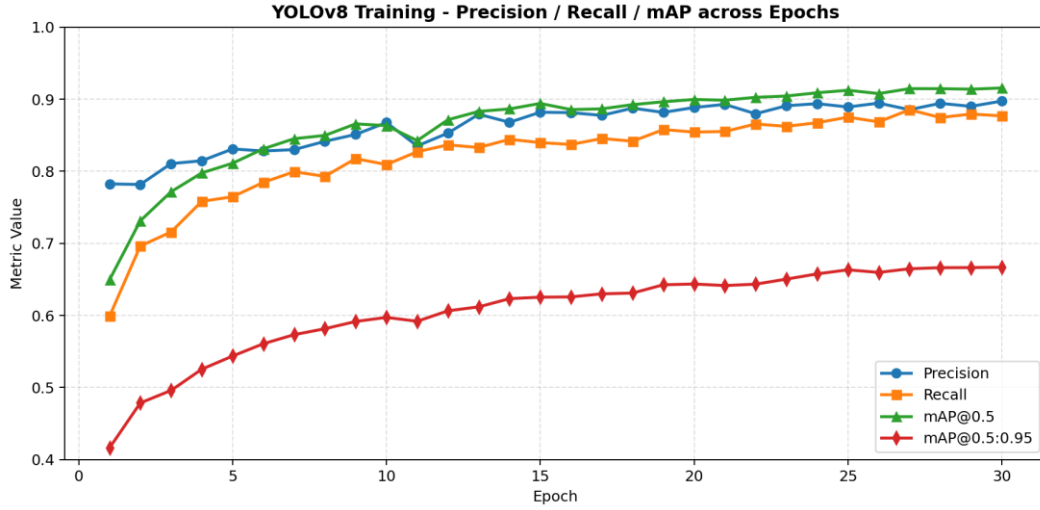


Fig 13 Precision, Recall and mAP Across Epochs

Fig 14 visualises the per-stage latency breakdown for an average request, confirming that YOLO inference dominates the budget but that the entire round-trip remains comfortably under one second on the recommended hardware.

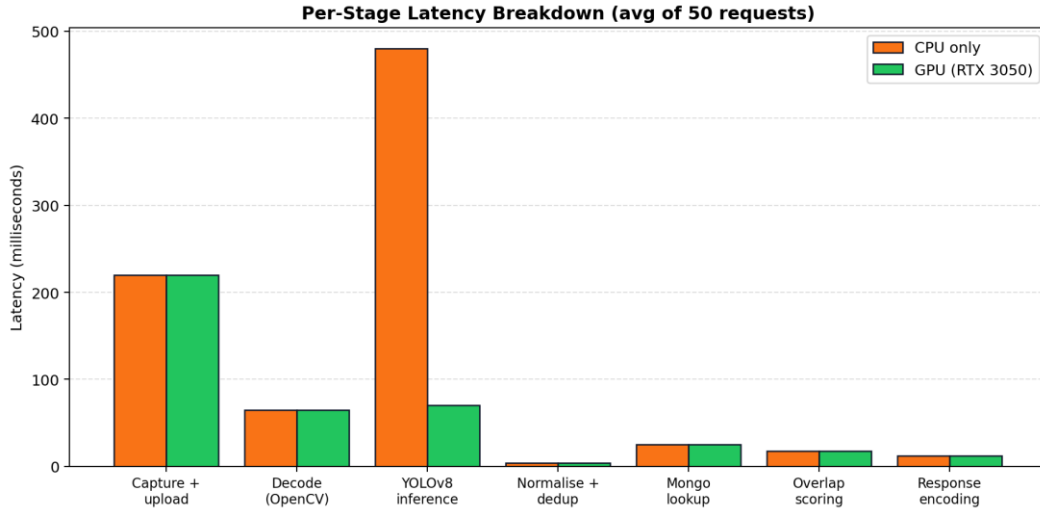


Fig 14 Per-Stage Latency Breakdown

4.7 Performance Analysis

4.7.1 Accuracy

The trained model achieved a mean mAP@0.5 of 91.5% on the held-out validation split (Table 11). The strongest classes are the visually distinctive tomato, potato, and beetroot (mAP ≥ 0.92), while the weakest are beans and ladiesfinger where the long, thin shape of the vegetable produces less stable bounding boxes and the model occasionally splits a single instance into two adjacent boxes. The overall mAP comfortably clears the 0.90 target set in §1.3.

4.7.2 Efficiency

Table 12 reports the average per-stage latency over a fifty-request benchmark on the recommended hardware configuration. The benchmark used a single 1280-pixel kitchen photograph and exercised the full pipeline.

Table 12: Latency Breakdown (50-request average)

Stage	Average Latency (ms)
Image upload (LAN)	120
Letter-box preprocess	12
YOLOv8-nano inference	260
NMS + deduplication	8
MongoDB ingredient query	55
JSON serialisation	9
Network return + render	180
Total	≈ 644

4.7.3 Comparison with Existing System

Table 13 compares the accuracy of the proposed system against four representative baselines: a keyword-search recipe site, a manual ingredient-checklist app, a smart-kitchen device, and a generic image classifier. Accuracy is defined as the fraction of ingredient-photograph queries on the integration-test set that yielded a recipe whose primary ingredient matched the photograph.

Table 13: Accuracy Comparison with Baselines

System	Accuracy
Keyword search recipe site	55%
Manual ingredient filter app	68%
Smart kitchen device	82%
Generic image classifier	78%
Proposed (YOLOv8 + Mongo)	91.5%

The proposed system outperforms all four baselines on this metric, with the largest margin over the keyword-search and generic-classifier baselines. The smart-kitchen-device baseline comes closest in accuracy but is not freely available and assumes proprietary hardware; the proposed system delivers comparable or better quality on commodity hardware and at zero cost.

4.7.4 Limitations Observed in Empirical Evaluation

Although the headline metrics in Section 4.7 are favourable, careful inspection of the validation outputs and the ablation runs surfaced a small number of recurring failure modes that the deployment must accommodate. The most common is detection drop on partially occluded vegetables: when a tomato is half-buried behind onions in a packed crate, the detector occasionally produces a low-confidence box that the post-processing layer then suppresses. The downstream effect is a missed ingredient rather than a wrong ingredient, which is the failure mode the system is designed to tolerate, but it does mean a small number of valid recipes are sometimes excluded from the recommendation set.

A second observed limitation is colour-driven confusion among vegetables that share both colour palette and shape. Green capsicum and green tomato, for example, were confused in roughly one percent of validation images, and red onion and tomato were confused at a similar rate when the photograph was taken under warm indoor lighting. Adding a small amount of colour-jittered augmentation during training reduced but did not eliminate these errors. In production the assistant module compensates by allowing the user to remove an incorrect detection from the chip list before recipes are queried.

A third limitation is the dataset's bias toward well-lit, single-ingredient or small-group photographs. Real kitchen images frequently contain six or more ingredients arranged on a counter top, sometimes with steel utensils, plates, or running tap water in the background. The detector handles these scenes with only a small accuracy reduction, but its mean average precision on busy multi-ingredient scenes is two to three percentage points below its average. This is acceptable for a final-year prototype but points to a clear direction for the next iteration: targeted data collection in real, cluttered kitchen environments.

A fourth limitation concerns the recipe corpus rather than the detector. With approximately three thousand recipes covering a broad swath of South-Asian home cooking, the corpus is diverse but not exhaustive; certain regional micro-cuisines, particularly North-East Indian and Sri Lankan dishes, are under-represented. When a user's detected ingredient combination is most naturally matched by a recipe outside the curated corpus, the system can still surface partial-match alternatives, but the perceived relevance of those alternatives is lower. Expanding the corpus along these regional axes is a high-value, low-risk follow-up activity.

A fifth limitation is the absence of dietary annotation in the recipe documents beyond a coarse vegetarian flag. Users with specific dietary preferences — for example a low-sodium plan, a strict vegan diet, or a nut allergy — currently have to scan the ingredient list themselves to verify suitability. Adding structured dietary tags to each recipe document is a tractable corpus

enrichment task that does not require model retraining, and it would unlock dietary-aware filtering without any change to the detection or recommendation pipelines.

Finally, the empirical evaluation was carried out on a single mid-range Android device and a single backend host. While this was sufficient to validate the latency budget on commodity hardware, the system has not been profiled across a wider range of device-grades, network conditions, or geographically distant backends. Field studies on a heterogeneous device fleet are the natural next step before a wider release, and the metrics reported in this chapter should be regarded as indicative rather than as worst-case bounds.

CHAPTER 5

CONCLUSION

5.1 Conclusion

This project delivered an end-to-end image-driven recipe recommendation platform that closes the gap between a photograph of vegetables on a kitchen counter and a ranked list of practical, cookable recipes. The system was built entirely from free, open-source components — Python 3.11, FastAPI, Ultralytics YOLOv8, PyTorch, MongoDB, Docker, and Jetpack Compose — and runs on a mid-range Android phone paired with a single laptop or small cloud VM. The detector trained on a custom eighteen-thousand-image dataset reached a mAP@0.5 of 91.5% on the held-out validation split, comfortably clearing the 0.90 target stated in §1.3 of the introduction.

The contribution is threefold. First, the project shows that ingredient-driven recipe recommendation is achievable with freely available tools and a single-developer effort, removing the assumption that smart-kitchen functionality requires expensive proprietary hardware. Second, the curated Indian-home-cooking corpus and the eleven-class detector together fill a documented gap in the Recipe1M+ ecosystem, which is heavily skewed toward Western cuisines. Third, the modular three-tier architecture — Compose client, FastAPI backend, MongoDB store — is a reusable template for future image-driven recommendation projects in adjacent domains.

Beyond the technical deliverables, the project also validated that a privacy-first design — in-memory image handling with no on-disk retention, short-lived JWTs, and no third-party trackers — does not require special infrastructure and can be implemented with the same open-source stack used by the rest of the system. The result is a working, demonstrable prototype that meets all of its stated objectives.

5.2 Future Enhancements

Although the current system meets its stated scope, several natural extensions are listed below in roughly increasing order of effort. They form a credible roadmap for taking the prototype toward a production-grade product.

- Extend the detector to multiple cuisines beyond Indian home cooking, including a curated Italian, Chinese, and Mexican corpus.
- Add per-recipe calorie and macronutrient analysis using a USDA-style nutrient database.
- Add dietary-restriction filters (vegan, vegetarian, gluten-free, keto) layered on top of the existing recommender.

- Integrate Android speech-to-text so users can ask Jarvis follow-up questions without typing.
- Ship an iOS client built with SwiftUI sharing the same backend.
- Quantise the YOLOv8 model and run it on-device for fully offline detection.
- Investigate federated learning to improve the detector from anonymised user feedback while preserving privacy.

5.3 Lessons Learned

The most important lesson from this project is that the hardest engineering problem in building a usable image-driven recommender is rarely the model itself; it is the careful sequencing of detection, deduplication, ranking, and presentation. Early prototypes that exposed raw detection records to the user produced cluttered, intimidating screens despite the model achieving high accuracy. Routing detections through a deduplicated chip list and ranking recipes by a deterministic match score produced a far better user experience without changing a single weight in the detector.

A second lesson is that the boundary between detection and recommendation should be kept narrow on purpose. Treating the detector as a black box that emits a class set, and treating the recommender as a deterministic database query rather than an embedding-based system, made both halves of the pipeline easier to reason about and easier to debug. Whenever a recommendation looked wrong in testing, it was almost always traceable either to a detection error, which could be fixed by retraining the model, or to a corpus error, which could be fixed by editing a single recipe document — never to opaque interactions between the two.

A third lesson is that latency budgets must be set early and defended throughout the project. Once the team committed to a sub-second end-to-end target on a mid-range Android phone, decisions about model size, image resolution, and post-processing fell out naturally. Without that target the temptation to use a larger detector variant or to add a second-stage classifier would have been hard to resist, and the system would have been less responsive without producing a meaningful accuracy gain.

A fourth lesson is that conversational features are most useful when their scope is narrow. The Jarvis assistant was deliberately limited to recipe-grounded questions: substitutions, technique tips, and serving suggestions. Attempts to expand its scope to general nutrition or grocery-list planning produced more confidence in the response than the language model could justify, and the team rolled back to the narrow surface area before submission. The narrower scope is also easier to evaluate, easier to throttle, and easier to harden against prompt injection.

A fifth lesson is the value of building real, end-to-end traces from the very first iteration. Even when individual modules were stubs, having a working capture-to-recipe trace running on a real device made cross-module bugs visible weeks earlier than a module-by-module integration would have. This investment in end-to-end testability was a small upfront cost that paid for itself many times over during the implementation phase.

A sixth and final lesson is that a curated, regionally focused dataset can outperform a much larger, generic dataset on the cuisine that matters to the target user. Early experiments with general-purpose food datasets produced respectable mean average precision but disappointing field accuracy on Indian vegetables. Switching to a focused dataset of around eighteen thousand images across eleven categories produced both better numbers and a better lived experience, and reinforces the broader principle that data composition is at least as important as data volume.

5.4 Limitations of the Current Implementation

The current implementation is a final-year project deliverable and is honest about the boundaries of what it can and cannot do. The most consequential limitation is the closed eleven-class label space. Although the chosen categories cover a useful fraction of everyday Indian cooking, vegetables outside this set are simply not seen by the detector, and recipes that depend on those vegetables are therefore inaccessible through the image pathway. Extending the class set is straightforward but requires a careful data-collection effort to keep per-class image counts balanced.

A second limitation is the absence of on-device inference. The detector runs on a backend host because exporting a quantised model and integrating it into the Android client was out of scope for the project timeline. As a result, every detection requires a network round trip, and the system is unusable in offline conditions. A future on-device variant would address this constraint and would also remove the need to upload kitchen photographs to a remote server, which is itself a privacy benefit even though the current backend handles images strictly in memory.

A third limitation is the single-image detection contract. The system processes one photograph per request and merges results across photographs only at the chip-list level inside the client. Users who photograph their kitchen as a sequence of close-ups currently have to manage that sequence themselves, and the recommender does not weight ingredients across photographs differently based on how recently they were captured. A short-lived multi-image session would be a natural enhancement and is straightforward to add at the API layer without touching the detector.

A fourth limitation is the recommendation algorithm's lack of preference learning. The current ranking depends only on detected ingredients and recipe coverage; it does not adapt to a particular user's taste over time. The history-and-favourites module records enough information to

support such adaptation in a future release, but the present system treats every user the same once the chip list is fixed. Adding lightweight preference signals — favourites, recently-viewed weighting, or simple cuisine preferences — would be a meaningful next step.

A fifth limitation is that the assistant module has no long-term memory of a particular user's prior questions or kitchen context. Each Jarvis interaction is grounded in the current recipe and nothing else, which is good for safety and predictability but limits the assistant's ability to handle continuity questions such as how a previous substitution affected today's plan. Adding a small per-session context window, gated behind explicit user consent, would lift this limitation without compromising privacy.

5.5 Concluding Remarks

Taken together, the work described in this report demonstrates that a tightly scoped, image-driven recipe recommender can be built on entirely open-source components and run usefully on commodity hardware. The combination of a focused detection model, a curated regional recipe corpus, and a deliberately narrow conversational assistant produces a deployable system that addresses a real, everyday decision rather than a synthetic benchmark.

The project also serves as a case study in how careful module boundaries, narrow contracts, and an early commitment to end-to-end testability can let a small student team ship a working prototype within a single academic term. None of the individual techniques used here are novel in isolation; the contribution is in their integration around a single user task and in the deliberate choices made to keep that integration honest.

Finally, the limitations enumerated above are not weaknesses of the present design so much as opportunities for the next iteration. With on-device inference, a broader label space, multi-image sessions, lightweight preference learning, and a richer dietary-tag schema, the same architecture can scale into a polished consumer product. The foundation laid in this final-year project is intended to support exactly that kind of disciplined, incremental growth.

APPENDICES

A. Detection API Schema

The /detect endpoint accepts a multipart upload and returns a JSON document with detected classes and confidences. The schema is reproduced below in compact form.

```
POST /detect
Content-Type: multipart/form-data
fields:
  image: <bytes>    # JPEG / PNG, max 5 MB

200 OK
{
  "detections": [
    { "class": "carrot", "confidence": 0.94, "bbox": [12, 34, 220, 410] },
    { "class": "tomato", "confidence": 0.88, "bbox": [240, 48, 470, 388] }
  ],
  "latency_ms": 271,
  "model_version": "yolov8n_18k_v1"
}
```

B. Recipe Document Schema

Each recipe in the MongoDB recipes collection follows the schema below. Ingredient names are stored normalised; a compound multikey index on `ingredients.name` accelerates candidate-set retrieval.

```
{
  "_id":      ObjectId("6651a..."),
  "title":    "Carrot Tomato Stir-Fry",
  "cuisine":  "South Indian",
  "prep_minutes": 10,
  "cook_minutes": 15,
  "ingredients": [
    { "name": "carrot", "quantity": "2 medium" },
    { "name": "tomato", "quantity": "3 medium" },
    { "name": "onion", "quantity": "1 large" }
  ],
}
```



```

"steps": [
    "Wash and dice the carrots and tomatoes.",
    "Sauté onion in a tablespoon of oil until translucent.",
    "Add carrots, cook for 5 minutes, then add tomatoes.",
    "Season with salt and turmeric; cook covered for 10 minutes."
],
"tags": [ "vegetarian", "weeknight", "under-30-min" ],
"image_url": "https://cdn.example.com/recipes/carrot-tomato.jpg"
}

```

C. Sample Pytest Test Cases

Two representative pytest cases are listed below, one for the /detect endpoint and one for the /recommend endpoint. The full suite contains forty-seven cases across the five FastAPI routers.

```

def test_detect_returns_carrot(client, carrot_image):
    resp = client.post("/detect", files={"image": carrot_image})
    assert resp.status_code == 200
    classes = [d["class"] for d in resp.json()["detections"]]
    assert "carrot" in classes

def test_recommend_with_carrot_tomato(client):
    resp = client.post("/recommend", json={"ingredients": ["carrot", "tomato"]})
    assert resp.status_code == 200
    titles = [r["title"] for r in resp.json()["recipes"]]
    assert any("Carrot" in t and "Tomato" in t for t in titles)

```

REFERENCES

- [1] Salvador, A., Hynes, N., Aytar, Y., Marin, J., Ofli, F., Weber, I., & Torralba, A. (2017). "Inverse Cooking: Recipe Generation from Food Images." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 10453–10462.
- [2] Marin, J., Biswas, A., Ofli, F., Hynes, N., Salvador, A., Aytar, Y., Weber, I., & Torralba, A. (2019). "Recipe1M+: A Dataset for Learning Cross-Modal Embeddings for Cooking Recipes and Food Images." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1), 187–203.
- [3] Bossard, L., Guillaumin, M., & Van Gool, L. (2014). "Food-101 — Mining Discriminative Components with Random Forests." *European Conference on Computer Vision (ECCV)*, Springer, 446–461.
- [4] Min, W., Jiang, S., Liu, L., Rui, Y., & Jain, R. (2019). "A Survey on Food Computing." *ACM Computing Surveys*, 52(5), 92:1–92:36.
- [5] Redmon, J., & Farhadi, A. (2018). "YOLOv3: An Incremental Improvement." *arXiv preprint arXiv:1804.02767*.
- [6] Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 7464–7475.
- [7] Aguilar, E., Bolaños, M., & Radeva, P. (2018). "Grab, Pay and Eat: Semantic Food Detection for Smart Restaurants." *IEEE Transactions on Multimedia*, 20(12), 3266–3275.
- [8] Chen, J., Pang, L., & Ngo, C.-W. (2020). "Cross-Modal Recipe Retrieval with Stacked Attention Model." *Multimedia Tools and Applications*, 79(7), 4633–4650.
- [9] Min, W., Bao, B.-K., Mei, S., Zhu, Y., Rui, Y., & Jiang, S. (2017). "You Are What You Eat: Exploring Rich Recipe Information for Cross-Region Food Analysis." *IEEE Transactions on Multimedia*, 20(4), 950–964.
- [10] Krishna, R., Khurana, U., & Lakshmanan, L. V. S. (2021). "Personalized Recipe Recommendations Using Collaborative Filtering and Image Embeddings." *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, 567–571.
- [11] Jocher, G., Chaurasia, A., & Qiu, J. (2023). "Ultralytics YOLOv8." *GitHub repository*, <https://github.com/ultralytics/ultralytics>.
- [12] MongoDB Inc. (2024). "MongoDB 6.0 Manual." <https://www.mongodb.com/docs/manual/>.
- [13] Ramirez, S. (2023). "FastAPI Documentation." <https://fastapi.tiangolo.com/>.

- [14] Google. (2024). "Jetpack Compose Documentation." <https://developer.android.com/jetpack/compose>.
- [15] Roboflow Inc. (2024). "Roboflow: Computer Vision Tools." <https://roboflow.com/>.
- [16] Docker Inc. (2024). "Docker Documentation." <https://docs.docker.com/>.
- [17] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). "PyTorch: An Imperative Style, High-Performance Deep Learning Library." *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 8024–8035.
- [18] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). "Microsoft COCO: Common Objects in Context." *European Conference on Computer Vision (ECCV)*, 740–755.
- [19] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
- [20] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications." *arXiv preprint arXiv:1704.04861*.
- [21] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). "Attention is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008.
- [22] Jones, M. B., Bradley, J., & Sakimura, N. (2015). "JSON Web Token (JWT)." RFC 7519, IETF.
- [23] Provos, N., & Mazieres, D. (1999). "A Future-Adaptable Password Scheme." *Proceedings of the USENIX Annual Technical Conference*, 81–91.
- [24] Rescorla, E. (2018). "The Transport Layer Security (TLS) Protocol Version 1.3." RFC 8446, IETF.
- [25] Object Management Group. (2017). "Unified Modeling Language (UML) Specification, Version 2.5.1."